ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR
Membre de HONORIS UNITED UNIVERSITIES

HONORIS UNITED UNIVERSITIES

Rapport de projet PFA

2024/2025

Filière ingénierie informatique et réseaux.

Sous le thème :

Plateforme Intelligente de Réservation et de Gestion d'Espaces de Coworking

Réalisé par :

KHAIROUNI Achraf

ROBAII Ikram

Encadré par :

Pr. EL BOUHADI Ouafae

Pr. BENYOUSSEF marwa

DEDICACE

A nos très chères mères,

Quoi que nous fassions ou disions, nous ne saurons point vous remercier comme il se doit.
Votre affection nous couvre, votre bienveillance nous guide, et votre présence à nos côtés a toujours été notre source de force pour affronter les différents obstacles.

A nos très chers pères,

Vous avez toujours été à nos côtés pour nous soutenir et nous encourager. Que ce travail traduit notre gratitude et notre affection.

A nos chers frères et sœurs,

Puisse Dieu vous donne santé, bonheur, courage et surtout réussite.

A tous ceux que nous aimons et ceux qui nous aiment.

REMERCIEMENTS

Tout d'abord, nous tenons à remercier tout particulièrement et à témoigner toute notre reconnaissance aux personnes suivantes, pour leur dévouement et leur soutient dans la concrétisation de ce projet ingénieur :

Je tiens à exprimer ma profonde gratitude à mon encadrant, Ouafae Elbouhadi, pour son précieux soutien, ses conseils éclairés et son engagement tout au long de ce projet. Son expertise et sa bienveillance ont grandement contribué à mon apprentissage et à l'aboutissement de ce travail.
Merci pour votre patience et votre accompagnement inestimable.

L'EMSI , SMARTiLAB et l'ensemble des enseignants pour leur coopération professionnelle tout au long de cette expérience et pour avoir partagé avec nous, une partie de leurs savoir- faire et de leurs expériences professionnelles.

RÉSUMÉ

Le présent travail s'inscrit dans le cadre d'un projet d'ingénieur visant à concevoir et réaliser une plateforme web complète et évolutive pour la réservation et la gestion d'espaces de coworking. L'objectif est de fournir une solution centralisée qui répond aux besoins de trois types d'utilisateurs distincts : les clients finaux, les administrateurs de sociétés de coworking et les directeurs (CEO) de ces sociétés.

La plateforme se distingue par son architecture multi-rôles et ses fonctionnalités avancées :

Gestion des rôles et permissions : Un système hiérarchique (Client, Admin, CEO) où chaque rôle dispose de droits spécifiques, garantissant une gestion sécurisée et déléguée des opérations.

Système financier intégré : Les utilisateurs gèrent leurs réservations via un solde prépayé, rechargeable par des codes. Les transactions créditent automatiquement les comptes des propriétaires de salles.

Logique de réservation avancée : Le système gère les disponibilités en temps réel, en appliquant des règles métier complexes comme une durée minimale de réservation et un temps de battement obligatoire entre les créneaux.

Écosystème étendu et innovant :

Au-delà du site web, le projet intègre conceptuellement :

- Un **assistant IA** pour le support client.
- Des **notifications via WhatsApp (Twilio)**.
- Une **application de bureau Java** pour la gestion administrative.
- Un **système IoT (RFID/ESP32)** pour l'automatisation du check-in.

Notifications transactionnelles : Un système d'emailing robuste informe les utilisateurs de toutes les actions critiques (création de compte, confirmation, annulation, alertes de sécurité).

Développée avec Python et le framework Django, l'application est conçue pour être déployée sur **Microsoft Azure**, garantissant scalabilité, sécurité et performance pour répondre aux exigences d'un marché moderne.

ABSTRACT

This project report details the design and implementation of a comprehensive and scalable web platform for a coworking space reservation and management system. The primary objective is to deliver a centralized solution that addresses the distinct needs of three user roles: end-user clients, coworking company administrators, and their respective CEOs.

The platform is distinguished by its multi-role architecture and advanced features:

Role-Based Access Control: A hierarchical system (Client, Admin, CEO) where each role is granted specific permissions, ensuring secure and delegated operational management.

Integrated Financial System: Users manage their reservations through a prepaid balance, rechargeable via unique codes. Transactions automatically credit the accounts of the space owners, establishing a complete financial cycle.

Advanced Booking Logic: The system manages real-time availability, enforcing complex business rules such as a minimum one-hour booking duration and a mandatory 30-minute buffer between reservations for maintenance.

Extended and Innovative Ecosystem: Beyond the core web application, the project's architecture conceptually integrates:

- **An AI-powered assistant** for enhanced customer support.
- **WhatsApp notifications** via the Twilio API.
- **A Java-based desktop application** for administrative management.
- **An IoT system (RFID/ESP32)** for automated check-ins.

Transactional Notifications: A robust email system keeps users informed of all critical events, including account creation, booking confirmations, cancellations, and security alerts.

Developed using **Python** and the **Django** framework, the application is designed for deployment on **Microsoft Azure**, ensuring the scalability, security, and performance required to meet the demands of a modern digital marketplace.

LISTE DES ABREVIATIONS

API : Application Programming Interface

CEO : Chief Executive Officer

CRUD : Create, Read, Update, Delete

CSS : Cascading Style Sheets

DB : Database (Base de Données)

DRF : Django Rest Framework

HTML : HyperText Markup Language

HTTP : HyperText Transfer Protocol

IoT : Internet of Things (Internet des Objets)

ORM : Object-Relational Mapping

PaaS : Platform as a Service

RFID : Radio-Frequency Identification

SDK : Software Development Kit

SMTP : Simple Mail Transfer Protocol

SQL : Structured Query Language

UML : Unified Modeling Language

.env : Fichier de configuration pour les variables d'environnement.

TABLE DE FIGURE

(Logo Python et Django)..... 30
(Logo PostgreSQL) 31
(Logos HTML5, CSS3, JS)..... 31
(Logo Pillow et Azure Blob Storage)..... 32
(Logo Microsoft Azure)..... 32

9 TABLE DES MATIERES

DEDICACE 2

REMERCIEMENTS 3

RÉSUMÉ 4

ABSTRACT 5

Role-Based Access Control: 5

Integrated Financial System: 5

Advanced Booking Logic: 5

Extended and Innovative Ecosystem: 5

Transactional Notifications: 5

LISTE DES ABREVIATIONS 6

TABLES DES FIGURES 7

INTRODUCTION GENERALE 9

CHAPITRE 1 : PRESENTATION ET CONTEXTE GENERAL DU PROJET 11

1. Introduction : 11

2. Cahier des Charges 11

4. Étude de l'Existant 14

5. CONCLUSION 16

CHAPITRE 2 : ANALYSE ET CONCEPTION 18

1. Introduction : 18

2. Analyse des Besoins 18

a) Besoins Fonctionnels 18

b) Besoins Non Fonctionnels 19

3. Architecture Globale de la Solution 20

4. Conception de la Base de Données 22

5. Conclusion 28

CHAPITRE 3 : RÉALISATION 30

1. Introduction : 30

2. Environnement Logiciel 30

3. Intégration des Fonctionnalités Clés 34

4. Processus de Développement 34

INTRODUCTION GENERALE

L'émergence de nouvelles formes de travail, accélérée par la transformation numérique, a positionné les espaces de coworking comme des acteurs incontournables de l'écosystème professionnel moderne. Offrant flexibilité, collaboration et services mutualisés, ces environnements dynamiques répondent aux besoins des freelances, des startups et même des grandes entreprises. Cependant, la gestion de ces espaces présente des défis complexes, notamment en matière de réservation des salles, de gestion des accès, de facturation et de communication avec les clients. Les solutions existantes, souvent fragmentées ou rigides, peinent à offrir une expérience intégrée et fluide pour l'ensemble des parties prenantes.

C'est dans ce contexte que s'inscrit notre projet, qui vise à concevoir et développer une **plateforme web centralisée et intelligente pour la réservation et la gestion d'espaces de coworking**. L'objectif principal est de proposer une solution complète qui non seulement automatise et simplifie les processus de réservation pour les clients, mais qui fournit également aux propriétaires et gestionnaires de ces espaces des outils puissants pour optimiser leurs opérations et maximiser leur rentabilité.

Pour répondre à cette problématique, notre plateforme repose sur une architecture multi-rôles, distinguant les **Clients**, les **Administrateurs** et les **Directeurs (CEO)** de sociétés, chacun disposant d'une interface et de fonctionnalités adaptées à ses responsabilités. Le système intègre un modèle économique basé sur un solde prépayé, une logique de réservation avancée incluant la gestion des disponibilités en temps réel, et un système de notifications transactionnelles robuste pour maintenir une communication transparente avec les utilisateurs.

Au-delà de ces fonctionnalités fondamentales, notre vision pour ce projet s'étend à la création d'un véritable écosystème technologique. Nous avons ainsi conceptualisé l'intégration de briques innovantes telles qu'un **assistant conversationnel basé sur l'intelligence artificielle** pour le support client, des **notifications via WhatsApp**, une **application de bureau dédiée** pour la gestion administrative intensive, et un **système IoT avec technologie RFID** pour automatiser les contrôles d'accès et le suivi de présence.

Ce rapport détaillera l'ensemble du processus de développement de ce projet, de l'analyse des besoins à la réalisation technique. Le premier chapitre présentera le contexte général, le cahier des charges et les objectifs. Le deuxième chapitre sera consacré à l'analyse et à la conception, avec une description de l'architecture logicielle, de la modélisation de la base de données et des diagrammes UML. Enfin, le troisième chapitre abordera la phase de réalisation

CHAPITRE 1 : PRESENTATION ET CONTEXTE GENERAL DU PROJET

CHAPITRE 1 : PRESENTATION ET CONTEXTE GENERAL DU PROJET

1. Introduction :

Le présent travail s'inscrit dans le cadre d'un projet d'ingénieur visant à concevoir et réaliser une plateforme web complète et évolutive pour la réservation et la gestion d'espaces de coworking. L'objectif est de fournir une solution centralisée qui répond aux besoins de trois types d'utilisateurs distincts : les clients finaux, les administrateurs de sociétés de coworking et les directeurs (CEO) de ces sociétés.

2. Cahier des Charges

Le cahier des charges définit l'ensemble des fonctionnalités et exigences que doit satisfaire la plateforme de réservation. Parmi les points essentiels, on retrouve :

Gestion des Rôles et des Permissions :

L'application est fondée sur une architecture multi-rôles distinguant trois types d'utilisateurs : les **Clients**, les **Administrateurs** et les **CEO**. Un système de permissions strictes garantit que chaque utilisateur n'accède qu'aux fonctionnalités qui lui sont dédiées. Les CEO peuvent gérer leurs sociétés, leurs salles et leurs administrateurs, tandis que ces derniers supervisent les réservations, et les clients interagissent avec la plateforme pour réserver des espaces.

Authentification Sécurisée et Cycle de Vie du Compte :

Le système assure une gestion complète et sécurisée du compte utilisateur. À l'inscription, un processus de **vérification par email** est déclenché, envoyant un lien d'activation unique pour valider l'adresse et activer le compte. Les mots de passe sont systématiquement hachés avant leur stockage en base de données pour garantir leur confidentialité. L'authentification repose sur un système de sessions sécurisées géré par Django.

Système Économique Basé sur un Solde Virtuel :

Toutes les transactions financières de la plateforme s'effectuent via un **solde prépayé**. Les utilisateurs peuvent recharger leur compte en utilisant des codes de recharge pré-générés. Les paiements des réservations sont instantanément débités de ce solde, et le montant est transféré sur le solde du CEO propriétaire de la salle, assurant ainsi un cycle financier complet et traçable.

Logique de Réservation Avancée :

Le cœur de la plateforme réside dans son module de réservation intelligent. Le système vérifie la disponibilité d'une salle en temps réel en tenant compte des réservations existantes, mais également en appliquant des règles métier spécifiques : une **durée minimale de réservation de une heure** et un **créneau d'entretien obligatoire de 30 minutes** après chaque réservation, rendant la salle indisponible durant ce laps de temps.

Gestion des Annulations et Remboursements :

Pour offrir une flexibilité aux utilisateurs, la plateforme intègre une politique d'annulation pour les réservations à venir. Plutôt qu'un crédit direct sur le solde, le remboursement est effectué par la **génération automatique d'un nouveau code de recharge unique**, d'une valeur équivalente au montant payé, qui est communiqué à l'utilisateur par email.

Notifications Transactionnelles par Email :

Un système de notification par email est intégré pour accompagner l'utilisateur à chaque étape clé. Des emails automatisés sont envoyés pour la confirmation de compte, la validation de réservation, l'alerte de solde insuffisant, la confirmation d'annulation, ainsi que pour des alertes de sécurité comme les nouvelles connexions.

Interface de Gestion Complète pour les CEO :

Les propriétaires de sociétés (CEO) disposent d'un tableau de bord dédié leur permettant d'effectuer des opérations CRUD (Créer, Lire, Mettre à jour, Supprimer) sur leurs salles. Ils peuvent également gérer les informations de leur société et uploader plusieurs images pour chaque espace, avec une gestion simple via une interface web dédiée.

Architecture Évolutive et Orientée Services :

La plateforme est conçue pour être le noyau d'un écosystème plus large. Son architecture prévoit l'intégration future de modules externes via une API REST, notamment un **chatbot IA** pour le support, des **notifications WhatsApp**, une **application de bureau Java** pour l'administration et un **système IoT (RFID)** pour l'automatisation du check-in.

3. Le projet de plateforme de réservation d'espaces de coworking poursuit plusieurs objectifs majeurs :

Développer une plateforme centralisée et multi-rôles :
L'objectif principal est de créer un écosystème unique capable de servir efficacement trois audiences distinctes (Clients, Administrateurs, CEO) avec des interfaces et des droits d'accès différenciés, éliminant ainsi le besoin d'outils multiples et fragmentés.

Garantir une gestion des réservations fiable et automatisée :
Assurer un système de réservation robuste qui gère les disponibilités en temps réel. L'implémentation de règles métier précises, comme le tampon de 30 minutes et la durée minimale, vise à automatiser la planification et à prévenir les conflits de créneaux, optimisant ainsi l'exploitation des salles.

Mettre en place un modèle économique sécurisé et transparent :
Instaurer un système financier basé sur un solde prépayé pour simplifier les transactions. L'objectif est de fournir un processus de paiement et de remboursement clair, traçable et sécurisé, tant pour les clients qui paient que pour les CEO qui perçoivent les revenus de leurs salles.

Offrir une autonomie complète aux gestionnaires d'espaces :
Permettre aux CEO une gestion totale de leurs ressources (salles, images, matériel) via un tableau de bord dédié. Cet objectif vise à leur donner le contrôle nécessaire pour présenter leurs offres de manière attractive et gérer leurs opérations sans intervention technique.

Assurer la sécurité et l'intégrité des données à tous les niveaux :
Protéger les informations sensibles par des mécanismes de sécurité robustes, incluant le hachage des mots de passe, la validation des comptes par email pour prévenir les inscriptions frauduleuses, et une gestion stricte des permissions pour se prémunir contre les accès non autorisés.

Concevoir une architecture évolutive et prête pour l'avenir :
Construire une application non seulement fonctionnelle, mais aussi techniquement prête à évoluer. L'architecture modulaire de Django et la conception orientée API ont pour objectif de faciliter l'intégration future de technologies innovantes comme l'IA, l'IoT et d'autres canaux de communication (WhatsApp), assurant ainsi la pérennité du projet.

4. Étude de l’Existant

L'analyse des solutions existantes pour la réservation d'espaces de coworking, qu'il s'agisse de grandes plateformes de marché (comme WeWork ou Regus) ou de logiciels de gestion dédiés, a révélé plusieurs points d'amélioration possibles et des opportunités d'innovation :

Flexibilité du Modèle Économique :

La plupart des plateformes fonctionnent sur des modèles d'abonnement ou de paiement direct par carte, ce qui peut manquer de flexibilité. Notre système de solde prépayé et de remboursement par code de recharge est peu courant et représente une alternative intéressante, notamment pour la gestion de la trésorerie et la fidélisation des clients.

Automatisation des Règles de Gestion Opérationnelle :

De nombreux systèmes permettent de bloquer des créneaux manuellement, mais l'application automatique de règles complexes comme un temps de battement obligatoire entre chaque réservation pour l'entretien n'est pas une fonctionnalité standard. Cela représente une opportunité pour optimiser la gestion logistique des espaces.

Architecture Multi-Acteurs Intégrée :

Les solutions existantes séparent souvent la plateforme de réservation grand public des outils de gestion internes. Offrir une plateforme unique où un utilisateur peut être à la fois client et, après validation, devenir CEO d'une société, avec des vues et des droits qui s'adaptent dynamiquement, constitue une approche plus intégrée et fluide.

Intégration de l'IoT pour l'Expérience sur Site :

L'expérience utilisateur s'arrête souvent à la réservation en ligne. L'intégration de technologies physiques comme des lecteurs RFID pour un check-in automatisé est encore rare. Ceci représente une opportunité majeure d'innovation pour fluidifier le parcours client et fournir des données d'occupation précises aux gestionnaires.

Interactivité et Support Client :

Le support client repose majoritairement sur des FAQ statiques, des formulaires de contact ou des lignes téléphoniques. L'intégration native d'un assistant IA capable d'interroger la base de données en temps réel pour répondre à des questions sur la disponibilité ou les équipements, ou encore l'utilisation de canaux directs comme WhatsApp, sont des axes d'innovation importants.

Notre projet se distingue donc par sa volonté de combiner ces fonctionnalités avancées. En proposant un modèle économique flexible, une automatisation poussée des règles de gestion, une architecture multi-rôles unifiée et une vision d'intégration avec l'IoT et l'IA, notre solution vise à offrir une plateforme plus complète, plus intelligente et mieux adaptée aux défis d'un environnement de coworking moderne.

Tableau Comparatif des Solutions Existantes

Application	Utilisateurs	Interface	Avantages	Limites
WeWork	Très large public : freelances, startups, grandes entreprises (clients finaux).	Moderne, très orientée marketing et image de marque. Principalement une vitrine pour leurs propres espaces.	Marque très forte, large réseau mondial, design des espaces attractif, événements et communauté.	Plateforme fermée (ne vend que ses propres espaces), modèle d'abonnement rigide, peu de flexibilité pour des réservations ponctuelles.
Regus (IWG)	Similaire à WeWork, clientèle souvent plus corporate et traditionnelle.	Fonctionnelle et professionnelle, mais peut être perçue comme moins moderne. Orientée vers la recherche et la réservation simple.	Très grand réseau international, variété d'offres (bureaux, coworking, salles de réunion), fiabilité et standardisation des services.	Processus de réservation parfois lourd, tarification moins transparente, expérience utilisateur moins "communautaire" que WeWork.
LiquidSpace	Place de marché ouverte : met en	Interface de type "marketplace"	Grande variété d'espaces et de	Qualité des espaces et services

	relation des gestionnaires d'espaces indépendants avec des clients.	(similaire à Airbnb), axée sur la recherche par carte et la comparaison des offres.	prix, flexibilité (réservation à l'heure, au jour, au mois), permet aux petits acteurs d'être visibles.	hétérogène, expérience utilisateur dépend du gestionnaire local, moins de fonctionnalités de gestion intégrées.
Coworker.com	Similaire à LiquidSpace, très orientée vers les nomades digitaux et les voyageurs d'affaires.	Très visuelle, avec un fort accent sur les avis, les photos et les "tours" virtuels des espaces.	Base de données mondiale exhaustive, communauté d'utilisateurs active avec beaucoup d'avis, facilité de comparaison.	Agit principalement comme un annuaire et un intermédiaire, la gestion post-réservation est souvent limitée, commission sur les réservations.

5. CONCLUSION

Ce chapitre a donc présenté le contexte général, le cahier des charges détaillé et les objectifs stratégiques de notre projet de **plateforme de réservation d'espaces de coworking**. Il pose les fondations d'une solution web intégrée qui vise à redéfinir l'interaction entre les clients et les gestionnaires d'espaces.

En intégrant des mécanismes de gestion multi-rôles, un système économique basé sur un solde virtuel, et une logique de réservation avancée, la plateforme dépasse le cadre d'un simple site de réservation. Elle est conçue comme un véritable outil de gestion opérationnelle et financière. L'accent mis sur la sécurité, avec une vérification rigoureuse des comptes, et sur la communication, via un système de notifications complet, a pour but d'établir un environnement de confiance et d'efficacité.

Enfin, en envisageant dès sa conception l'intégration de fonctionnalités innovantes telles que l'assistant IA, les notifications WhatsApp et l'automatisation par IoT, ce projet jette les bases d'une plateforme non seulement fonctionnelle, mais aussi résolument tournée vers l'avenir, prête à offrir une expérience utilisateur optimale dans l'écosystème dynamique du travail moderne.

CHAPITRE 2 : ANALYSE ET CONCEPTION

CHAPITRE 2 : ANALYSE ET CONCEPTION

1. Introduction :

Ce chapitre détaille l'analyse des besoins et la conception de la **plateforme de réservation et de gestion d'espaces de coworking**. Ce projet, qui s'inscrit dans le cadre d'un cursus d'ingénierie en informatique et réseaux, vise à offrir une solution web centralisée, sécurisée et performante, qui structure les interactions complexes entre les différents acteurs du monde du coworking.

L'analyse et la conception constituent des étapes fondamentales pour garantir que l'application finale réponde non seulement aux attentes des utilisateurs, mais qu'elle modélise aussi correctement les processus métier spécifiques à ce domaine. Une attention particulière est portée à la définition des exigences pour chaque rôle – qu'il s'agisse des **clients** recherchant de la flexibilité, des **CEO** souhaitant optimiser la gestion de leurs biens, ou des **administrateurs** assurant le bon déroulement des opérations quotidiennes.

À travers une modélisation rigoureuse de la base de données, une architecture logicielle modulaire et l'utilisation de diagrammes UML, ce chapitre pose les fondations techniques sur lesquelles repose l'ensemble du projet. Il a pour but de traduire les exigences fonctionnelles et non fonctionnelles en une structure technique cohérente et évolutive, prête à intégrer des fonctionnalités avancées comme l'assistance par IA ou les systèmes IoT.

2. Analyse des Besoins

a) Besoins Fonctionnels

Les besoins fonctionnels décrivent les actions que le système doit être capable d'exécuter. Pour notre plateforme de coworking, les principales fonctionnalités identifiées sont regroupées par module :

Module de Gestion des Utilisateurs et des Rôles :

Création de Compte et Vérification : Le système permet l'inscription de nouveaux utilisateurs avec une vérification obligatoire par email pour activer le compte et garantir l'authenticité de l'adresse fournie.

Authentification et Profil : Les utilisateurs disposent d'un système de connexion sécurisé et d'un espace profil où ils peuvent consulter leurs informations personnelles et leur solde.

Gestion Hiérarchique des Rôles : Le système doit gérer trois rôles (Client, Admin, CEO) avec des permissions distinctes. Un super-administrateur doit pouvoir assigner le rôle de CEO, et un CEO doit pouvoir créer et gérer les comptes de ses propres administrateurs.

Module de Gestion des Espaces (pour les CEO) :

Gestion des Sociétés et des Salles : Un CEO doit pouvoir créer et mettre à jour les informations de sa société, ainsi qu'ajouter, modifier et supprimer les salles de coworking qui lui sont associées.

Gestion des Médias et du Matériel : Le système doit permettre aux CEO d'uploader plusieurs images pour chaque salle et de spécifier le matériel disponible (ex: projecteur, tableau blanc), en le sélectionnant depuis une liste prédéfinie.

Module de Réservation et Financier (pour les Clients) :

Recherche et Consultation : Les utilisateurs doivent pouvoir consulter la liste de toutes les salles disponibles, les filtrer et voir une page de détail complète avec photos, description et caractéristiques.

Processus de Réservation : Le système doit permettre de réserver une salle pour une date et une heure données, en appliquant des règles strictes de disponibilité (horaires d'ouverture, créneaux déjà pris, et temps de battement de 30 minutes).

Gestion du Solde et Paiement : Les utilisateurs doivent pouvoir recharger leur solde via des codes prépayés. Le paiement d'une réservation doit être automatiquement débité de ce solde.

Annulation et Remboursement : Le système doit permettre à un utilisateur d'annuler une réservation future et de recevoir en contrepartie un code de remboursement par email, d'une valeur équivalente au montant payé.

b) Besoins Non Fonctionnels

Les besoins non fonctionnels concernent les contraintes techniques, ergonomiques et de performance que le système doit respecter. Pour notre application, ils incluent :

Ergonomie et Accessibilité : L'interface web doit être intuitive, responsive et facile à utiliser sur différentes plateformes (desktop, mobile), afin de garantir une expérience utilisateur fluide, que ce soit pour une simple recherche ou pour des tâches de gestion complexes.

Sécurité et Confidentialité : La sécurité est une priorité, assurée par le hachage systématique des mots de passe, la protection contre les injections SQL via l'ORM de Django, une gestion rigoureuse des permissions par rôle, et la validation des entrées utilisateur pour prévenir les failles de sécurité courantes.

Performance et Scalabilité : Le système doit être capable de gérer un volume croissant de données (utilisateurs, salles, réservations) et de trafic. L'architecture est pensée pour un déploiement sur une infrastructure cloud comme Azure, permettant une mise à l'échelle horizontale et verticale. L'utilisation de tâches asynchrones (Celery) sera envisagée pour les opérations longues comme l'envoi d'emails.
Robustesse et Maintenabilité : Le code est structuré en applications Django distinctes et modulaires (users, company, reservations, finances), ce qui facilite la maintenance, les tests et l'évolution future du projet. Une gestion claire des erreurs informe l'utilisateur sans compromettre la stabilité du système.
Interopérabilité : L'application doit être conçue avec une API REST (via Django Rest Framework) en perspective, afin de permettre la communication future avec les autres briques de l'écosystème (application Java, chatbot, système IoT).

3. Architecture Globale de la Solution

Pour répondre aux exigences de flexibilité, de maintenabilité et d'évolutivité du projet, nous avons opté pour une **architecture orientée services**, avec l'application Django comme noyau central. Bien que développée initialement comme une application web monolithique, sa conception interne est modulaire et son exposition externe est pensée via une **API REST (Application Programming Interface)**. Cette approche permet de découpler le backend (la logique métier) des différents "clients" qui consomment ses services, qu'il s'agisse de l'interface web, d'applications externes ou de systèmes connectés.

L'application Django agit comme le **"cerveau" de l'écosystème**. Elle est responsable de :

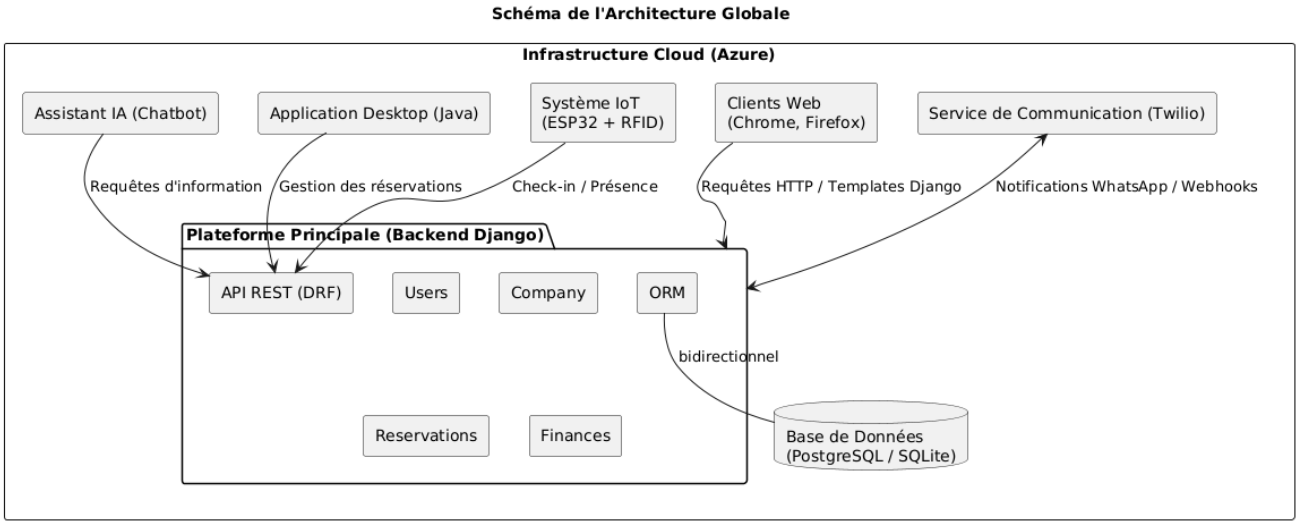
- La gestion de la logique métier (règles de réservation, calculs financiers).
- La sécurité et l'authentification des utilisateurs et des services.
- La persistance des données via son ORM (Object-Relational Mapper) qui communique avec la base de données.
- L'exposition de points d'accès (endpoints) sécurisés pour toutes les autres composantes du système.

Cette architecture se compose des briques suivantes :

1. **Le Cœur Applicatif (Backend Django) :** Il s'agit de l'application principale développée en Python avec le framework Django. Elle est elle-même subdivisée en applications modulaires (users, company, reservations, finances) pour une meilleure séparation des préoccupations. C'est elle qui héberge toute l'intelligence du système.

2. **La Base de Données :** Une base de données relationnelle **PostgreSQL** est choisie pour l'environnement de production en raison de sa robustesse, de sa fiabilité et de son support avancé des transactions, ce qui est crucial pour les opérations financières et de réservation. Pour le développement, SQLite est utilisé pour sa simplicité.
3. **La Façade Web (Frontend) :** C'est l'interface principale utilisée par les clients et les CEO. Elle est générée côté serveur par le moteur de templates de Django. Elle communique directement avec la logique interne de Django pour afficher les pages et traiter les formulaires.
4. **Les Services Externes et Clients :** Ce sont les composantes qui consomment les données et services du noyau Django via son API REST. Chaque service est autonome et spécialisé :
    - **Application de Bureau (Java) :** Destinée aux administrateurs pour une gestion intensive, elle interroge l'API pour afficher et modifier les réservations.
    - **Assistant Conversationnel (IA) :** Un service externe (potentiellement un microservice Python utilisant des modèles de langage) qui se connecte à des endpoints spécifiques de l'API pour récupérer des informations en temps réel (ex: "La salle X est-elle disponible demain à 10h ?").
    - **Service de Notification (Twilio) :** Le backend Django déclenche des appels à l'API de Twilio pour envoyer des notifications WhatsApp. Il peut également recevoir des requêtes (webhooks) de Twilio en réponse aux messages des utilisateurs.
    - **Système IoT (ESP32) :** Les microcontrôleurs installés dans les salles agissent comme des clients IoT. Lors d'un scan RFID, ils effectuent un appel sécurisé à un endpoint dédié de l'API pour enregistrer la présence d'un utilisateur.

Le schéma ci-dessous illustre les interactions entre ces différentes composantes.



4. Conception de la Base de Données

La conception de la base de données est une étape fondamentale qui conditionne la robustesse, la cohérence et la performance de l'application.

Nous avons adopté une approche **Model-First** grâce à l'ORM (Object-Relational Mapper) de Django, qui nous permet de définir la structure des données directement en Python via des classes (modèles).

L'ORM traduit ensuite ces modèles en un schéma relationnel dans la base de données, ce qui facilite la portabilité (de SQLite à PostgreSQL) et garantit la sécurité (prévention des injections SQL).

Le schéma a été conçu pour être normalisé, éviter les redondances et être optimisé pour les requêtes fréquentes, comme la recherche de disponibilité des salles ou la consultation des réservations d'un utilisateur.

4.1 Modèle Conceptuel de Données (MCD)

Le Modèle Conceptuel de Données ci-dessous illustre les principales entités du système et les relations entre elles.

Il met en évidence la centralité des entités **User** et **Salle**, qui sont au cœur de la plupart des interactions.

(Insérer ici votre schéma MCD réalisé avec draw.io, Looping Modeler, ou tout autre outil de modélisation.)

Relations entre les entités :

- **User (1,1) — (0,1) Societer**

  Un utilisateur peut être le CEO d'au plus une société ; une société est dirigée par un seul utilisateur (OneToOneField).
- **User (1,1) — (0,n) Reservation**

  Un utilisateur peut effectuer plusieurs réservations ; une réservation est effectuée par un seul utilisateur (ForeignKey dans Reservation).
- **User (1,1) — (0,n) RechargeSolde**

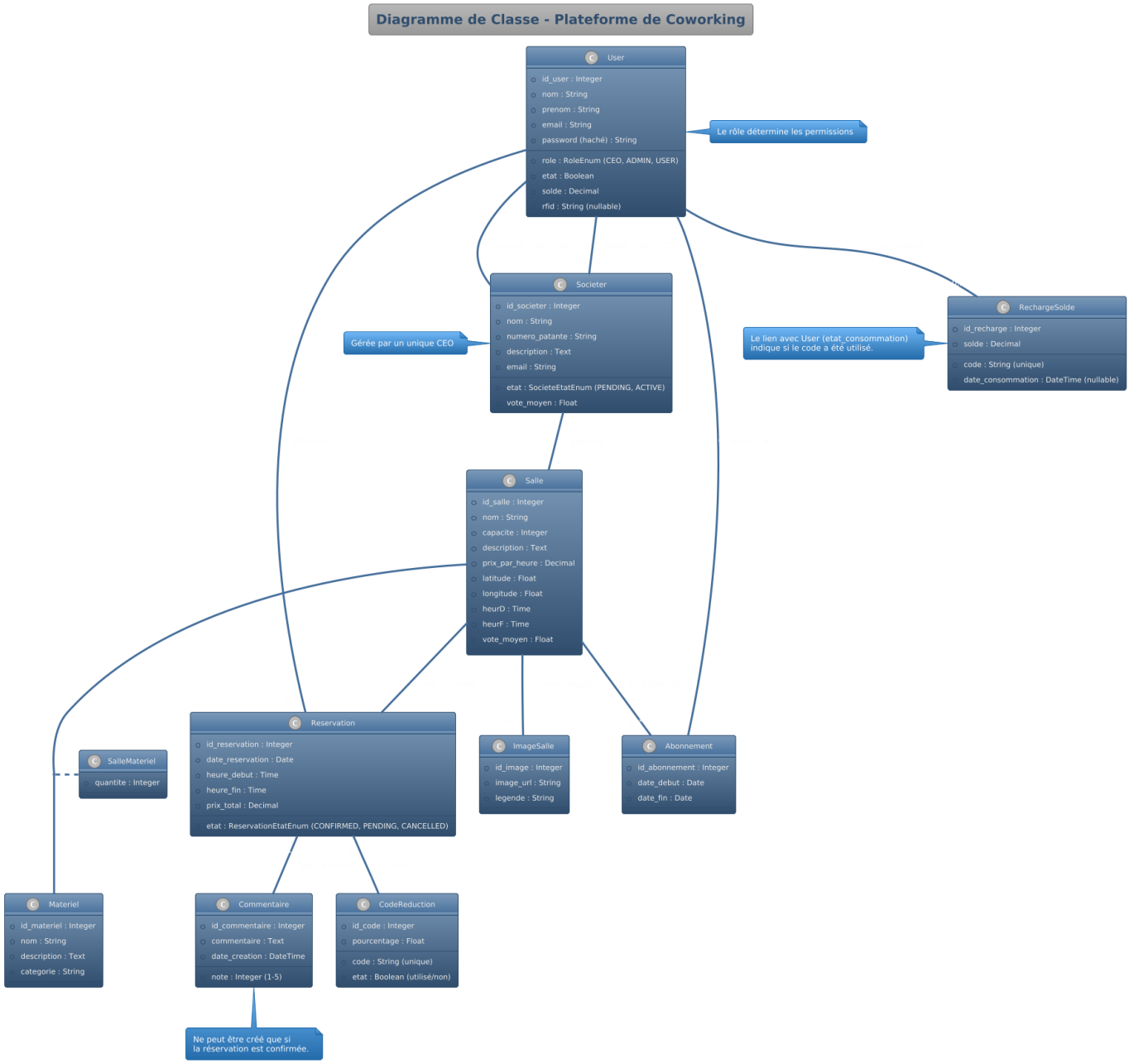
  Un utilisateur peut consommer plusieurs codes de recharge ; un code est utilisé par un seul utilisateur (ForeignKey dans RechargeSolde).
- **Societer (1,1) — (0,n) Salle**

  Une société peut posséder plusieurs salles ; une salle appartient à une seule société (ForeignKey dans Salle).
- **Salle (1,1) — (0,n) Reservation**

  Une salle peut avoir plusieurs réservations ; une réservation concerne une seule salle (ForeignKey dans Reservation).
- **Salle (1,1) — (0,n) ImageSalle**

  Une salle peut avoir plusieurs images ; une image est liée à une seule salle (ForeignKey dans ImageSalle).
- **Salle (0,n) — (0,n) Materiel**

  Une salle peut disposer de plusieurs types de matériel ; un matériel peut être dans plusieurs salles (ManyToManyField via une table de liaison Salle_Materiel).



4.2 Description des Modèles Principaux

Voici une description des modèles clés et de leurs champs importants :

User (Application users)

- Hérite de AbstractUser.
- **role** : CharField (choix entre CEO, ADMIN, USER). Définit les permissions sur la plateforme.
- **solde** : DecimalField (portefeuille virtuel de l'utilisateur, utilisé pour les paiements).
- **etat** : BooleanField (indique si le compte est activé par confirmation d'email).

- **rfid** : CharField (optionnel, identifiant unique RFID pour le système IoT).

Societer (Application company)

- **ceo** : OneToOneField vers User. Assure qu'une société a un seul CEO.
- **etat** : CharField (choix entre En attente, Active, Refusée). Géré par le super-administrateur.

Salle (Application company)

- **societer** : ForeignKey vers Societer.
- **prix_par_heure** : DecimalField (tarif de la salle à l'heure).
- **heurD, heurF** : TimeField (heures d'ouverture pour les validations de réservations).

Reservation (Application reservations)

- **salle** : ForeignKey vers Salle.
- **user** : ForeignKey vers User.
- **date_reservation**, **heure_debut**, **heure_fin** : champs temporels définissant le créneau réservé.
- **etat** : CharField (choix entre Confirmée, En attente, Annulée), suivi du cycle de vie de la réservation.

RechargeSolde (Application finances)

- **code** : CharField unique.
- **etat_consommation** : ForeignKey vers User (NULL si le code est encore disponible ; renseigné lorsqu'un utilisateur le consomme).

Diagramme de Cas d'Utilisation - Acteur Client

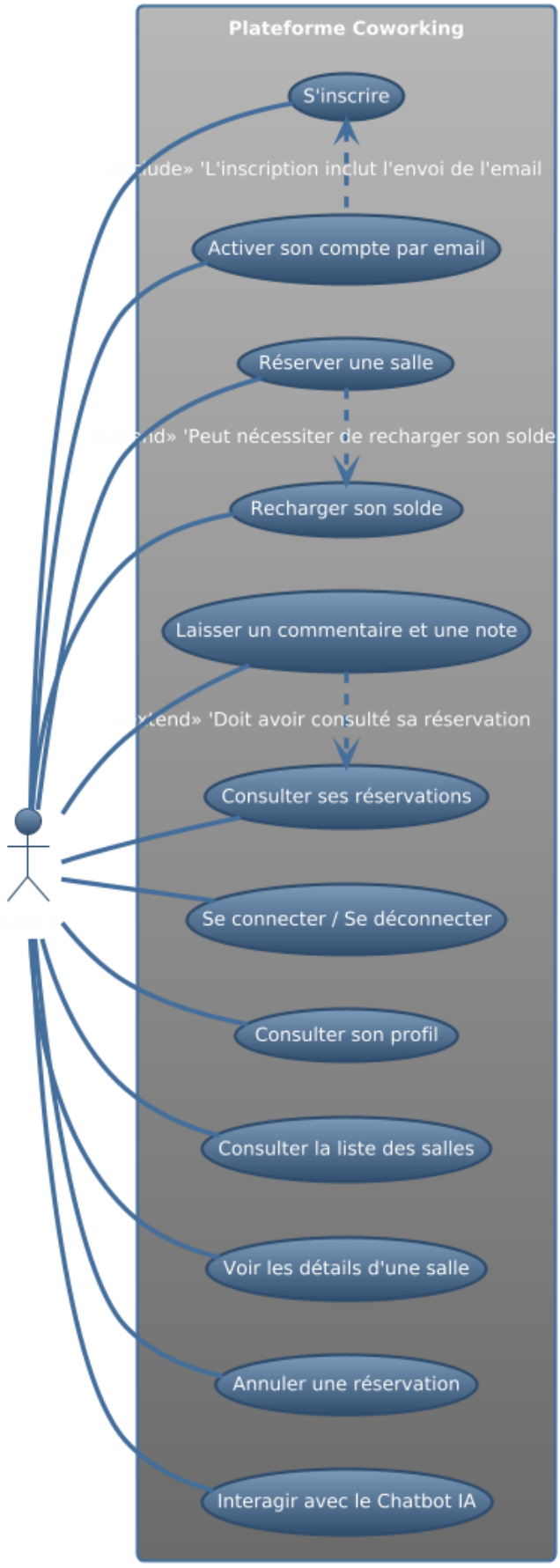
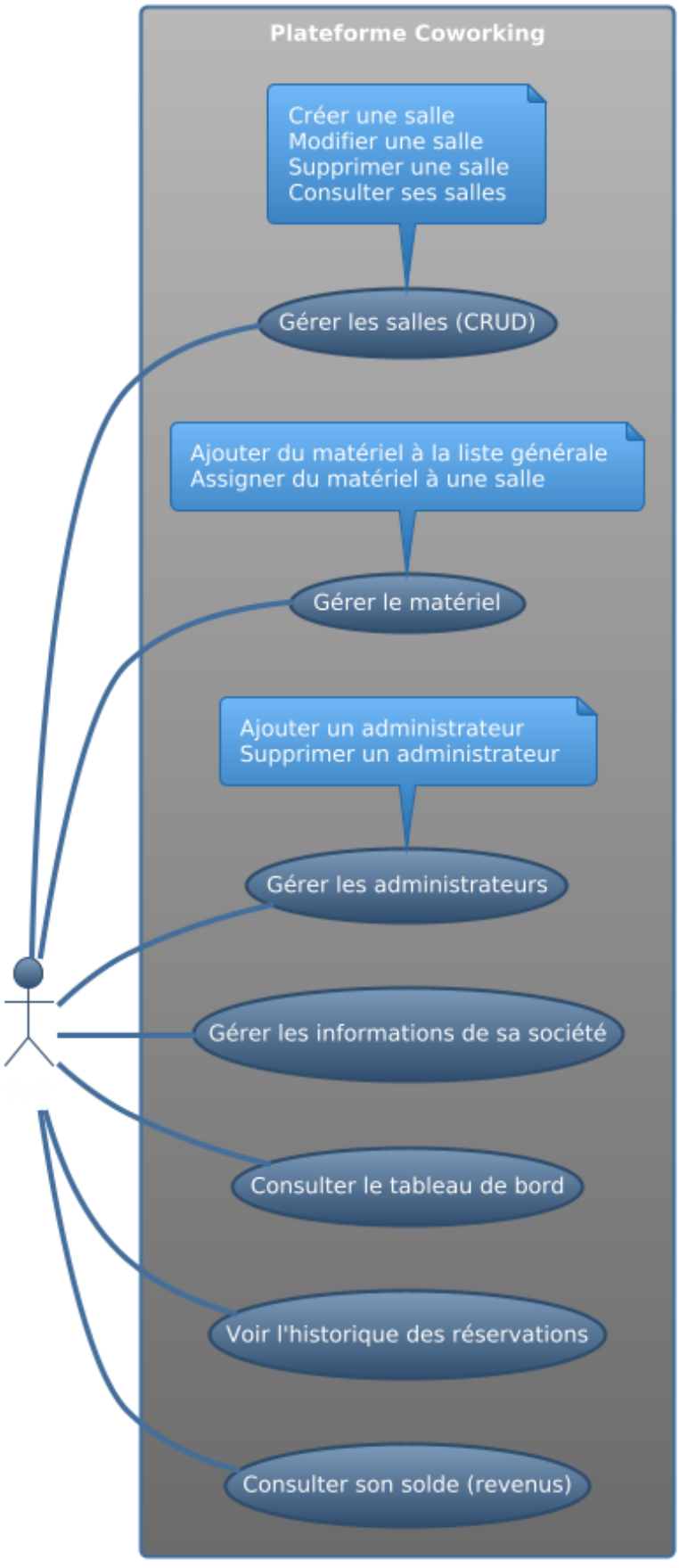
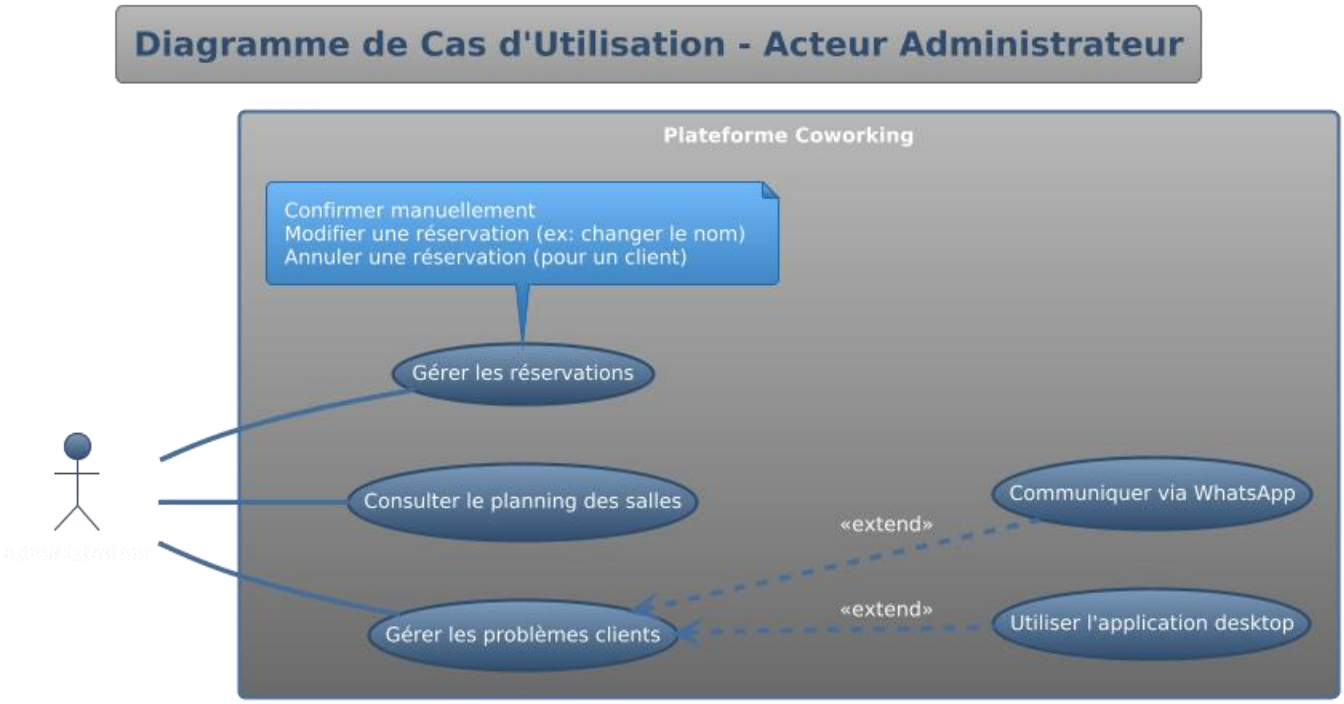


Diagramme de Cas d'Utilisation - Acteur CEO





5. Conclusion

Le chapitre d'analyse et de conception établit les fondations techniques du projet de **plateforme de coworking**. L'application vise à structurer un écosystème complexe en combinant une gestion multi-rôles, des processus métier automatisés et des mécanismes de sécurité robustes. La conception repose sur une modélisation rigoureuse de la base de données via l'ORM de Django et une architecture logicielle modulaire, garantissant un système évolutif et maintenable, prêt à répondre aux exigences variées des différents utilisateurs.

Ce chapitre prépare ainsi le terrain pour la phase de réalisation, en définissant clairement les fonctionnalités à implémenter, l'architecture technique à suivre, et les exigences de performance, de sécurité et d'ergonomie du projet.

CHAPITRE 3 : REALISATION

CHAPITRE 3 : RÉALISATION

1. Introduction :

La phase de réalisation traduit nos plans théoriques, issus de l'analyse et de la conception, en une plateforme de réservation d'espaces de coworking pleinement fonctionnelle. L'objectif de cette étape est de mettre en œuvre la logique métier complexe, de construire les interfaces utilisateur pour chaque rôle, et d'intégrer les différents modules techniques pour aboutir à une solution robuste, sécurisée et intuitive. Cette phase couvre l'implémentation technique, l'intégration des fonctionnalités clés et la mise en place d'un environnement de développement et de déploiement adapté aux exigences du projet.

2. Environnement Logiciel

Le choix des technologies et des outils de développement a été déterminant pour assurer la robustesse, la scalabilité et la sécurité de l'application. Voici les principaux éléments utilisés dans notre projet :

- **Langage de Programmation et Framework Backend :**
    - **Python et Django :** Le cœur de l'application est développé avec le langage Python et le framework Django. Ce choix est motivé par la rapidité de développement, la sécurité intégrée (protection CSRF, ORM anti-injection SQL), la structure modulaire ("apps") et un écosystème de bibliothèques très riche. Django gère l'authentification, la logique métier, le rendu des templates et la communication avec la base de données.



Figure 1(Logo Python et Django)

- **Base de Données :**
    - **SQLite (Développement) et PostgreSQL (Production) :** Pour la phase de développement, SQLite a été utilisé pour sa simplicité et sa légèreté. Pour la production, PostgreSQL est la

cible privilégiée en raison de sa performance, de sa robustesse pour les opérations transactionnelles complexes (gestion des soldes, réservations) et de sa parfaite intégration avec Django.



Figure 2 (Logo PostgreSQL)

- **Frontend et Interface Utilisateur :**
    - **HTML5, CSS3, JavaScript :** L'interface utilisateur est construite sur les standards du web, rendue côté serveur par le moteur de templates de Django. Cette approche assure une compatibilité maximale et une performance de chargement initiale rapide, sans nécessiter de framework JavaScript lourd côté client pour les fonctionnalités de base.



Figure 3(Logos HTML5, CSS3, JS)

- **Gestion des Fichiers et Médias :**
    - **Pillow :** Cette bibliothèque Python est essentielle pour le traitement des images (redimensionnement, validation) uploadées par les CEO pour leurs salles.
    - **Stockage Média :** En développement, les images sont stockées localement sur le serveur. Pour la production, l'architecture est conçue pour utiliser un service de stockage d'objets

comme **Azure Blob Storage** ou **Amazon S3** afin de garantir la scalabilité et la disponibilité des fichiers.



Figure 4(Logo Pillow et Azure Blob Storage)

- **Environnement et Déploiement :**
    - **Environnement Virtuel Python (venv) :** Utilisé pour isoler les dépendances du projet et garantir la reproductibilité de l'environnement de développement.
    - **Déploiement Cloud (Azure) :** L'application est conçue pour être déployée sur la plateforme **Microsoft Azure**, en utilisant des services managés comme **Azure App Service** pour l'hébergement de l'application Django et **Azure Database for PostgreSQL** pour la base de données.



Figure 5(Logo Microsoft Azure)

Tailwind CSS et DaisyUI :

Ces outils sont utilisés pour styliser l'application avec une interface moderne, ergonomique et entièrement responsive.



Figure 6(logo Tailwind CSS)

Cloudinary :

Pour l'upload et la gestion des images de profil ou des pièces jointes dans les messages.



Figure 7 (logo Cloudinary)

email :

Pour l’envoi des notifications



Figure 8(logo email)

3. Intégration des Fonctionnalités Clés

La réalisation de l'application repose sur l'intégration de plusieurs fonctionnalités avancées, détaillées ci-dessous :

Architecture Multi-Rôles :

Mise en place d'un système de permissions dynamique basé sur le champ role du modèle User. Des mixins de permissions ont été créés pour sécuriser l'accès aux vues et garantir que seuls les utilisateurs autorisés (ex: CEO) peuvent accéder aux fonctionnalités de gestion.

Logique de Réservation et de Disponibilité :

Implémentation de l'algorithme de vérification de disponibilité qui prend en compte non seulement les réservations existantes, mais aussi les heures d'ouverture de la salle et la règle du tampon de 30 minutes entre les réservations. Cette logique est au cœur du moteur de réservation.

Système Financier et Transactionnel :

Développement du cycle de vie financier complet : création de codes de recharge par l'admin, application de ces codes par l'utilisateur pour créditer son solde, débit du solde lors d'une réservation, crédit du solde du CEO correspondant, et remboursement par génération de code lors d'une annulation.

Gestion des Médias :

Intégration d'un système d'upload d'images permettant aux CEO d'associer plusieurs visuels à leurs salles, avec une gestion complète (ajout, suppression) via leur tableau de bord.

4. Processus de Développement

La phase de réalisation a suivi une méthodologie itérative et incrémentale, avec les étapes suivantes :

Mise en Place des Fondations :

Initialisation du projet Django, configuration de l'environnement, et définition des modèles de données de base (User, Societer, Salle).

Développement du Cycle de Vie Utilisateur :

Implémentation de l'inscription, de l'activation par email, de la connexion, du profil utilisateur et du système de recharge de solde.

Développement des Fonctionnalités de Gestion (CEO) :

Création du tableau de bord du CEO, et implémentation des fonctionnalités CRUD (Créer, Lire, Mettre à jour, Supprimer) pour la gestion des salles et de leurs images.

Implémentation du Cœur Métier (Réservation) :

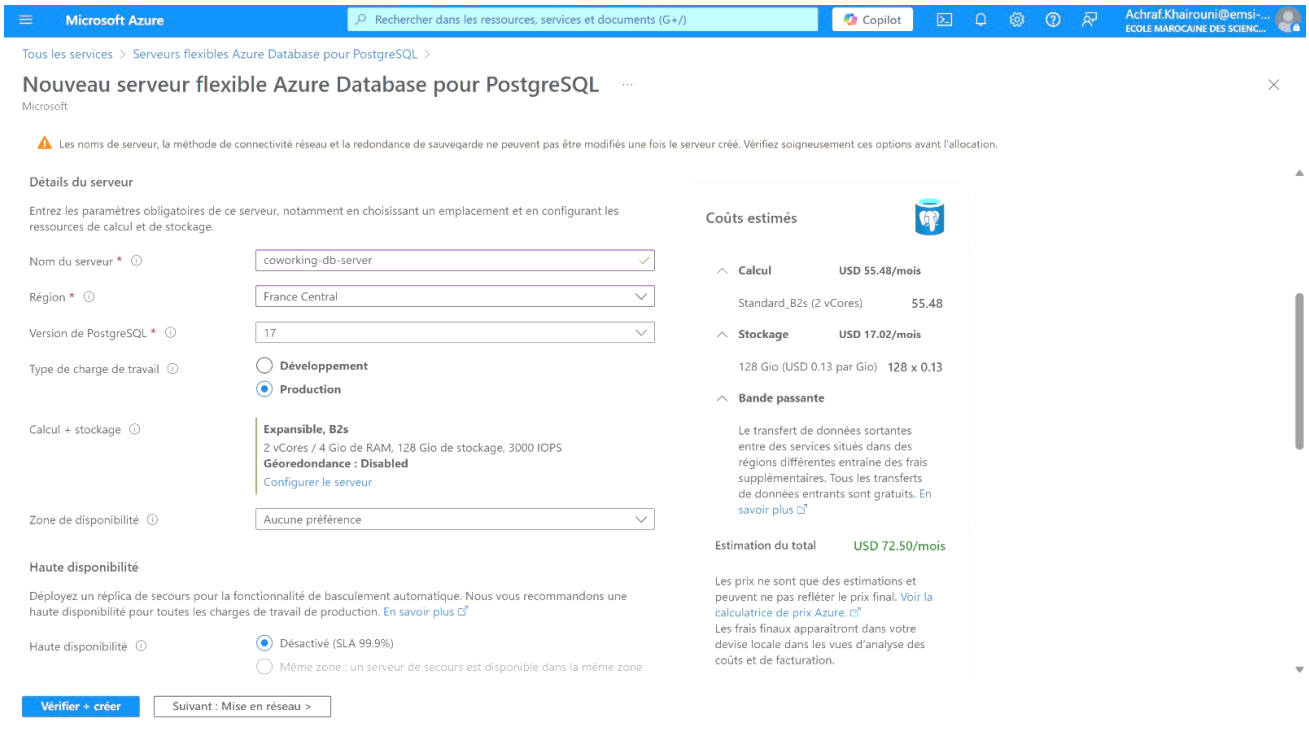
Développement du formulaire de réservation, de la logique de vérification de disponibilité et du processus de paiement et d'annulation.

Tests et Débogage Continus :

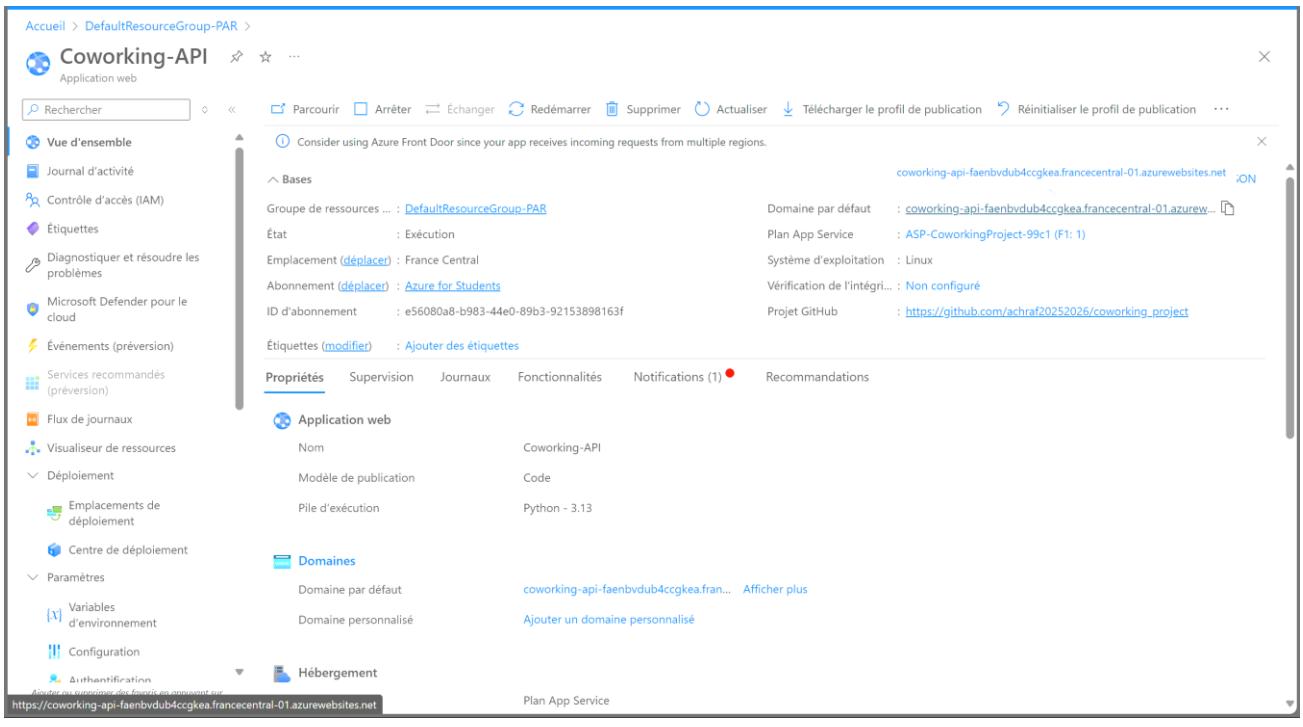
Après chaque étape, des tests manuels ont été effectués pour valider le bon fonctionnement de la fonctionnalité, la robustesse de la logique et la sécurité des accès.

5. Image démonstrative

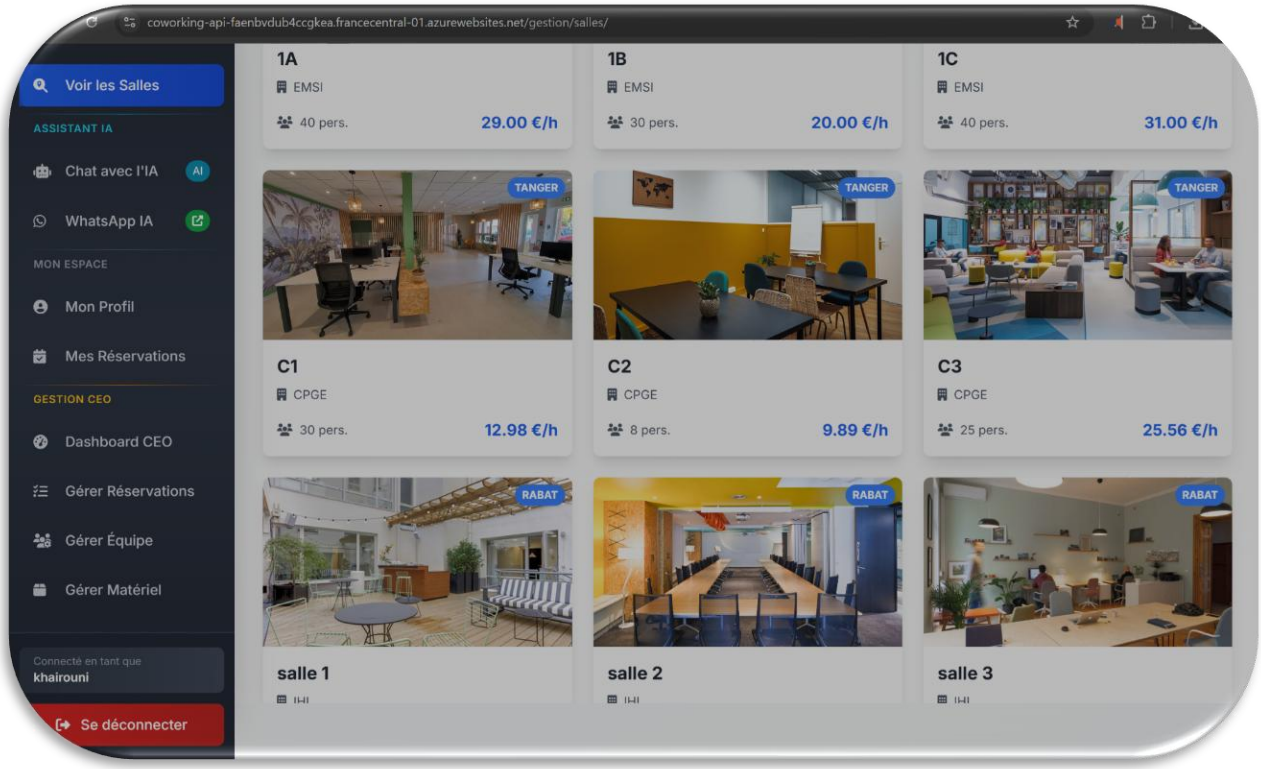
Base de donner azure postgreSQL



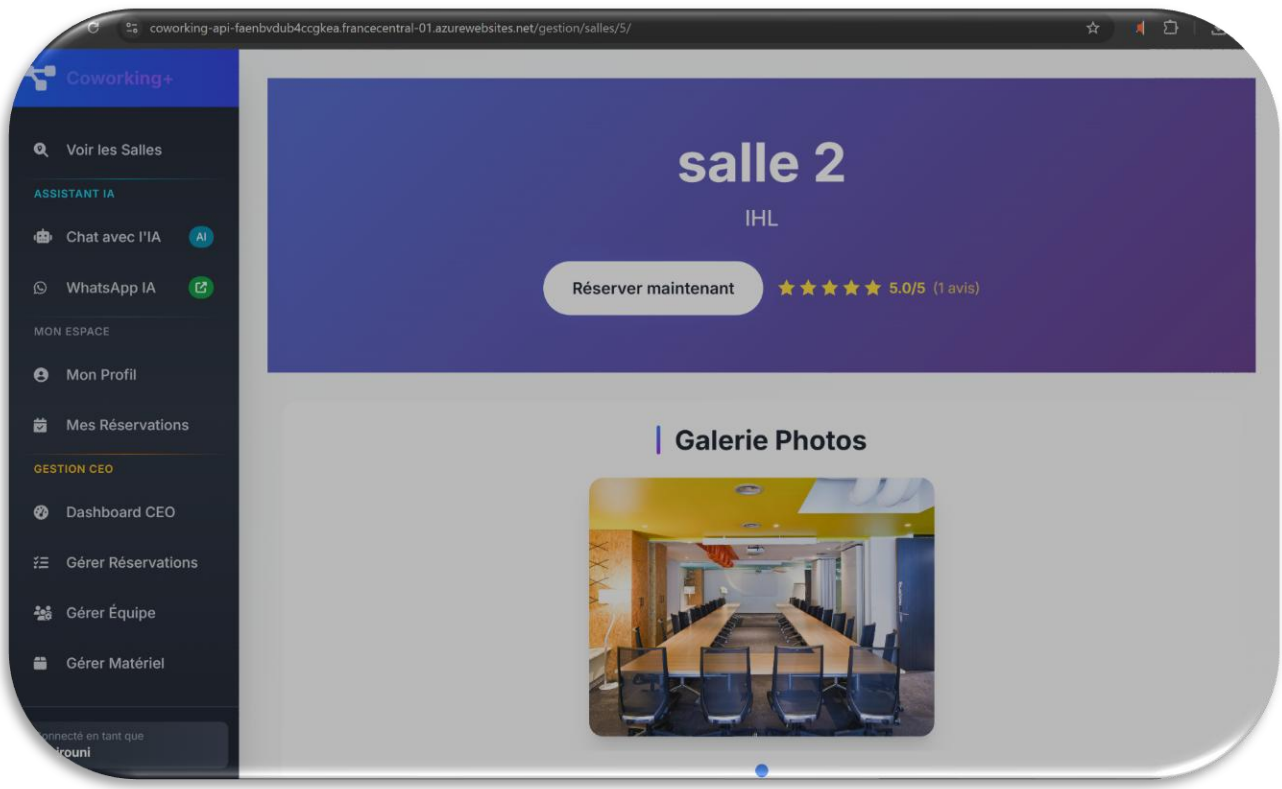
Hébergement sur serveur linux azure : nom du domaine : coworking-api-faenbvdub4ccgkea.francecentral-01.azurewebsites.net



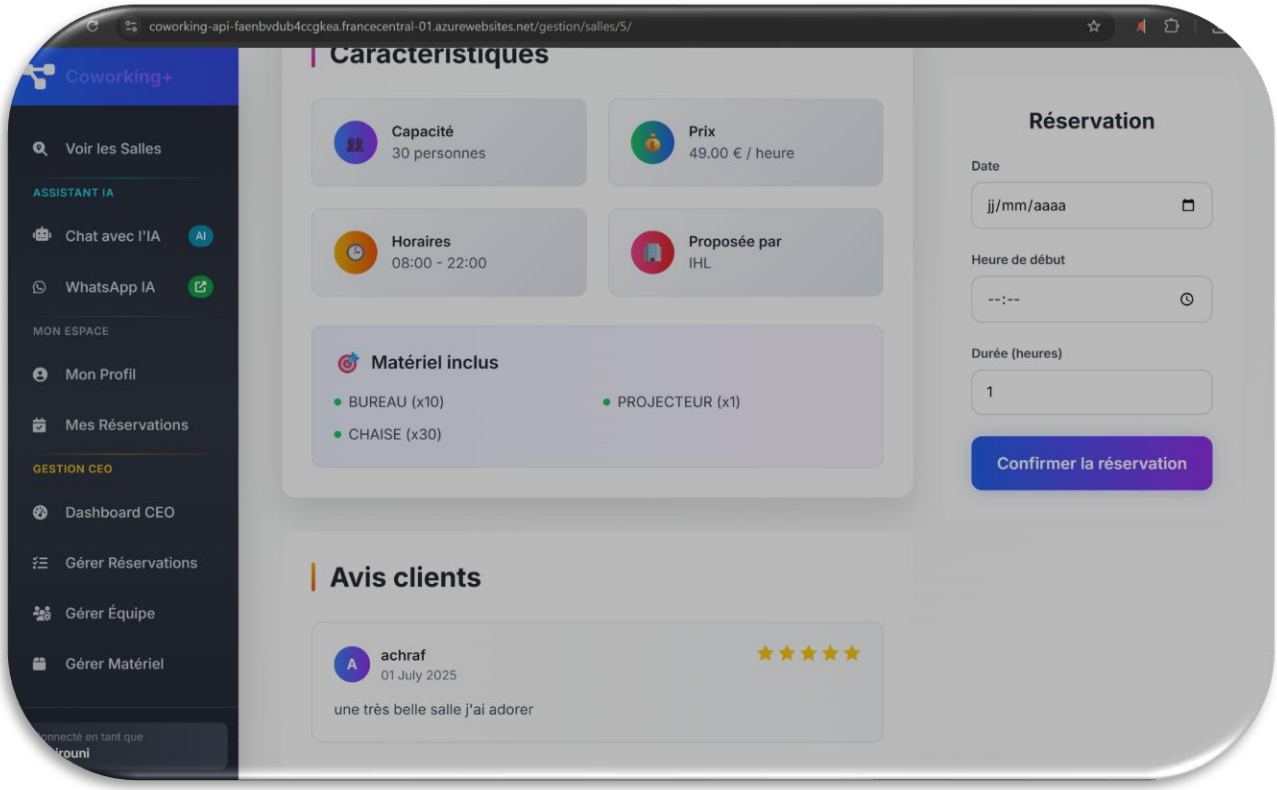
Catalogue des Salles Disponibles



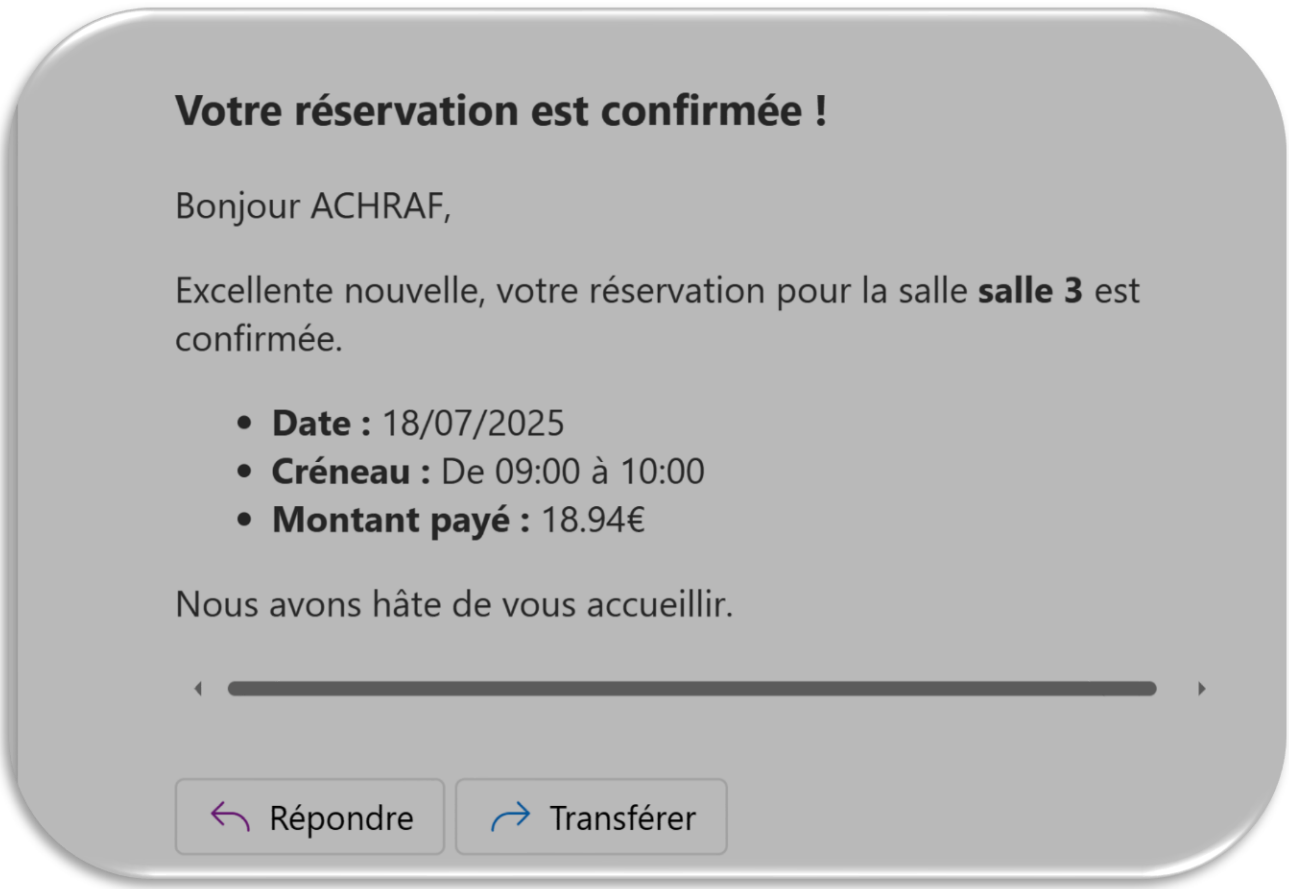
Page de Détail d'une Salle (En-tête)



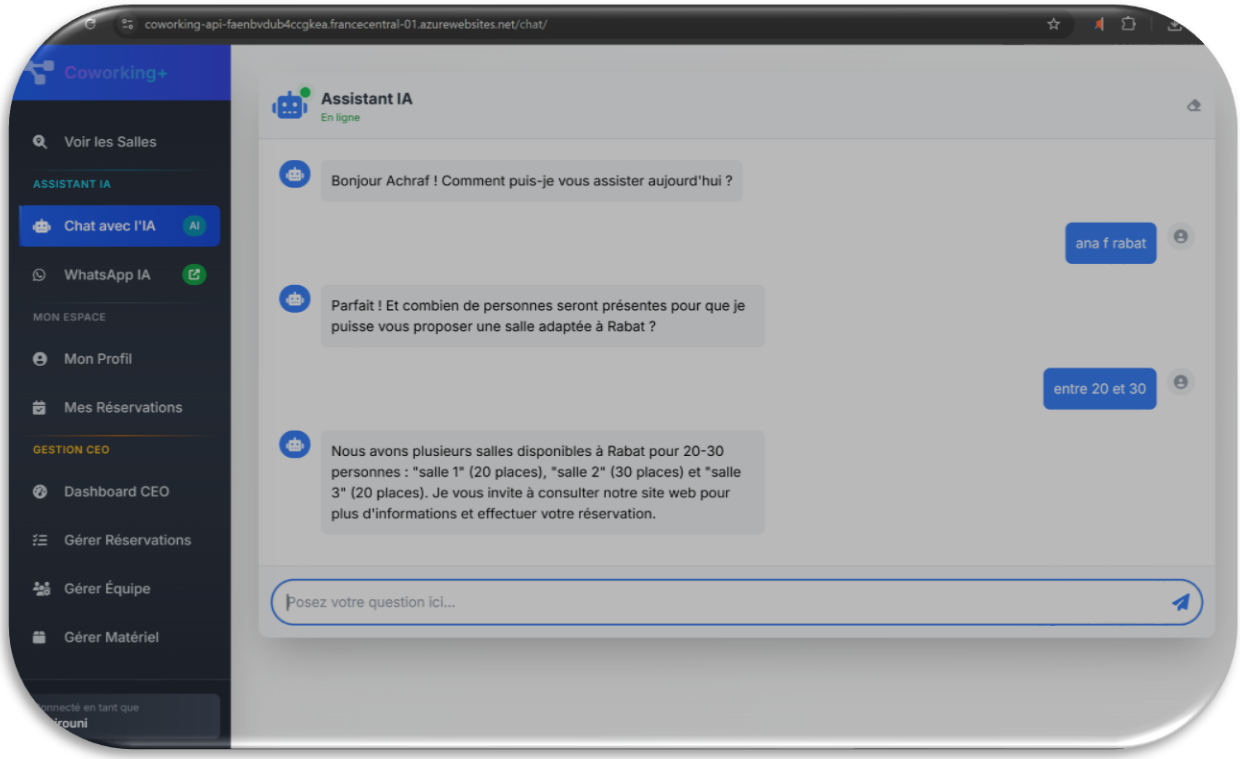
Détails, Avis et Formulaire de Réservation



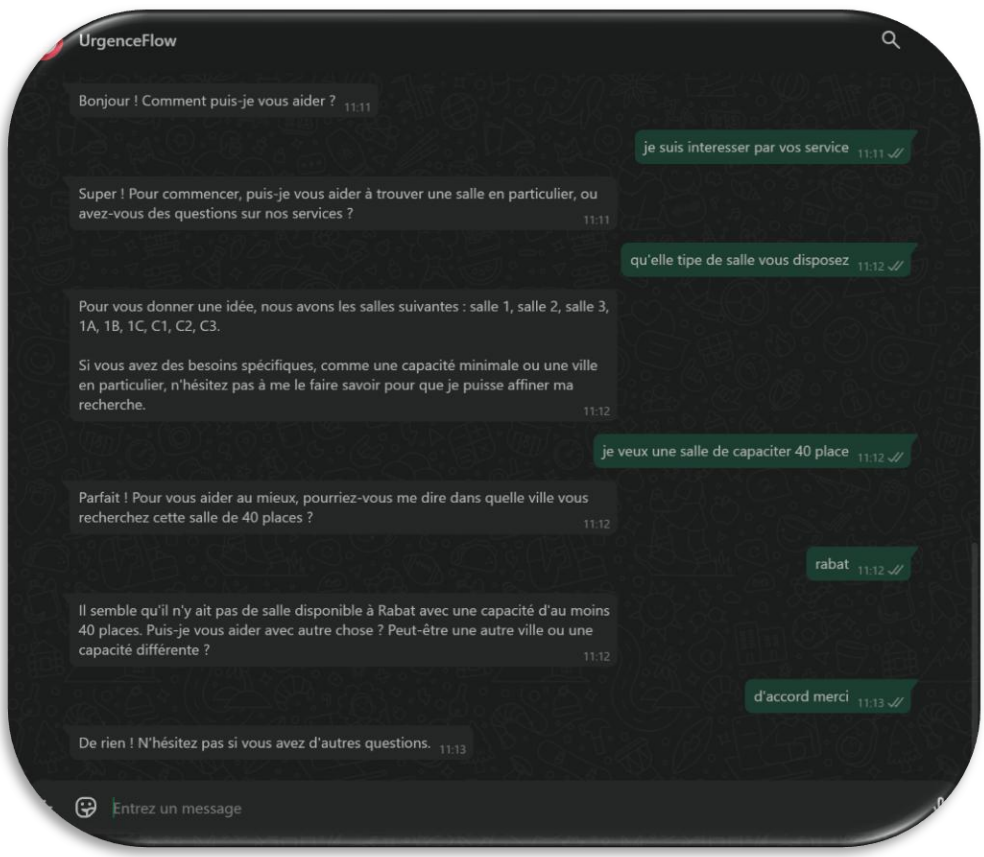
Email de Confirmation de Réservation



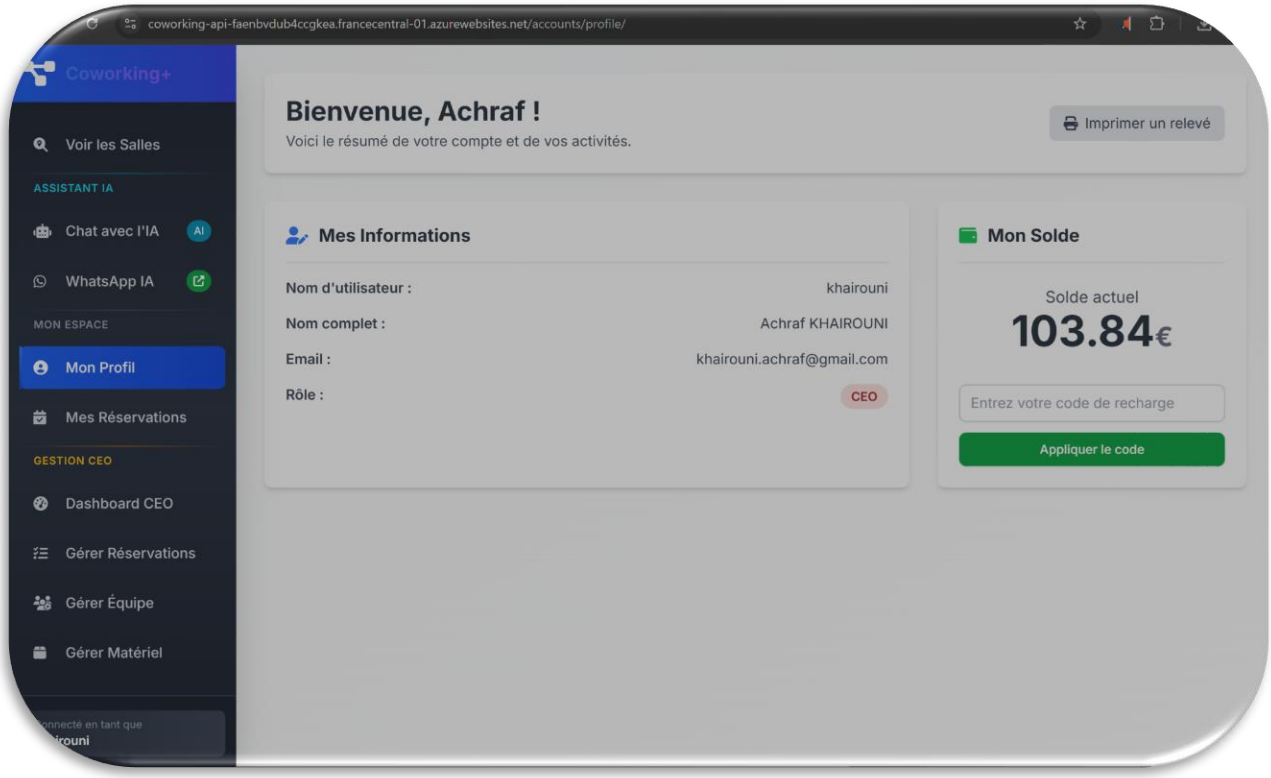
Assistant IA Conversationnel en Action



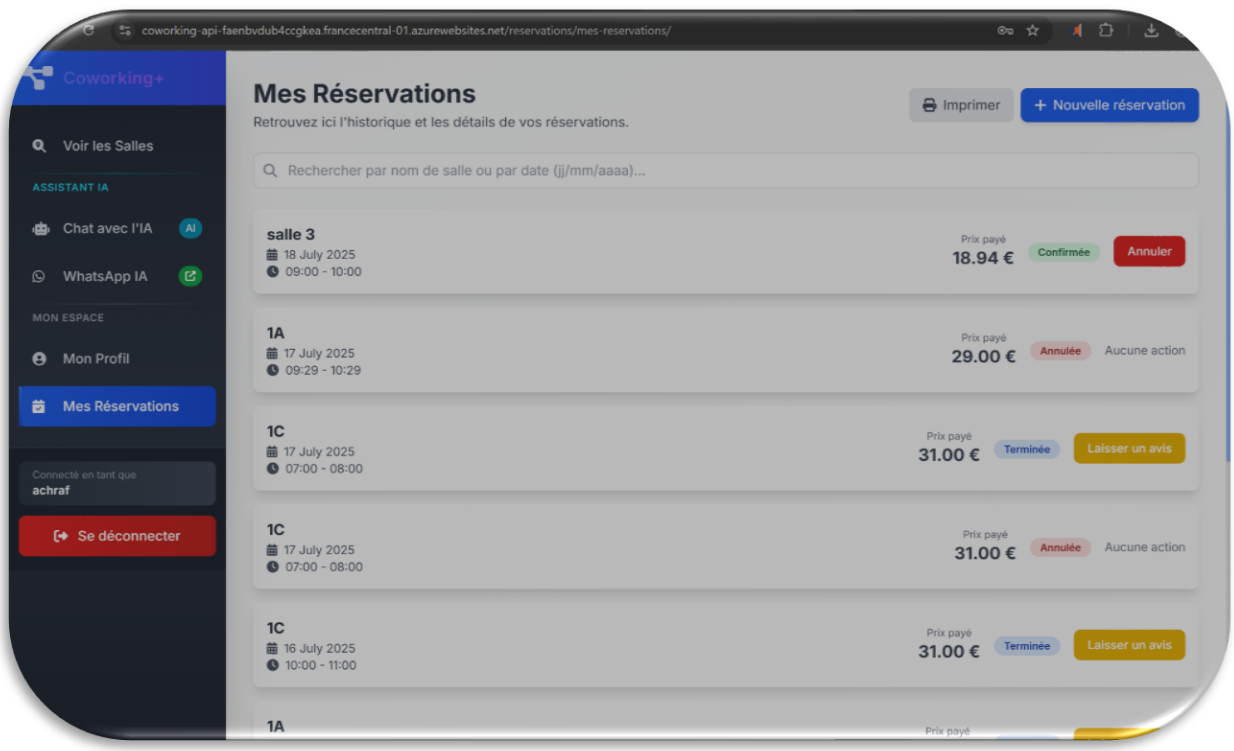
Canal de Support via WhatsApp



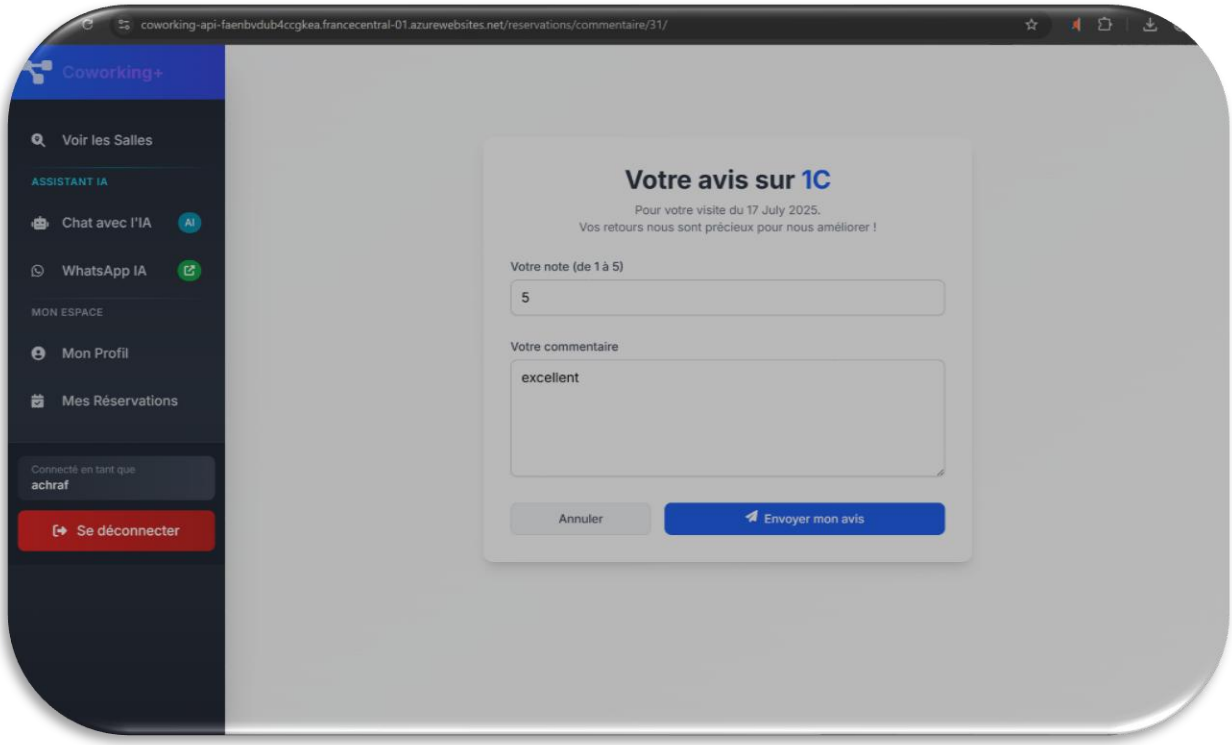
Profil Utilisateur et Gestion du Solde



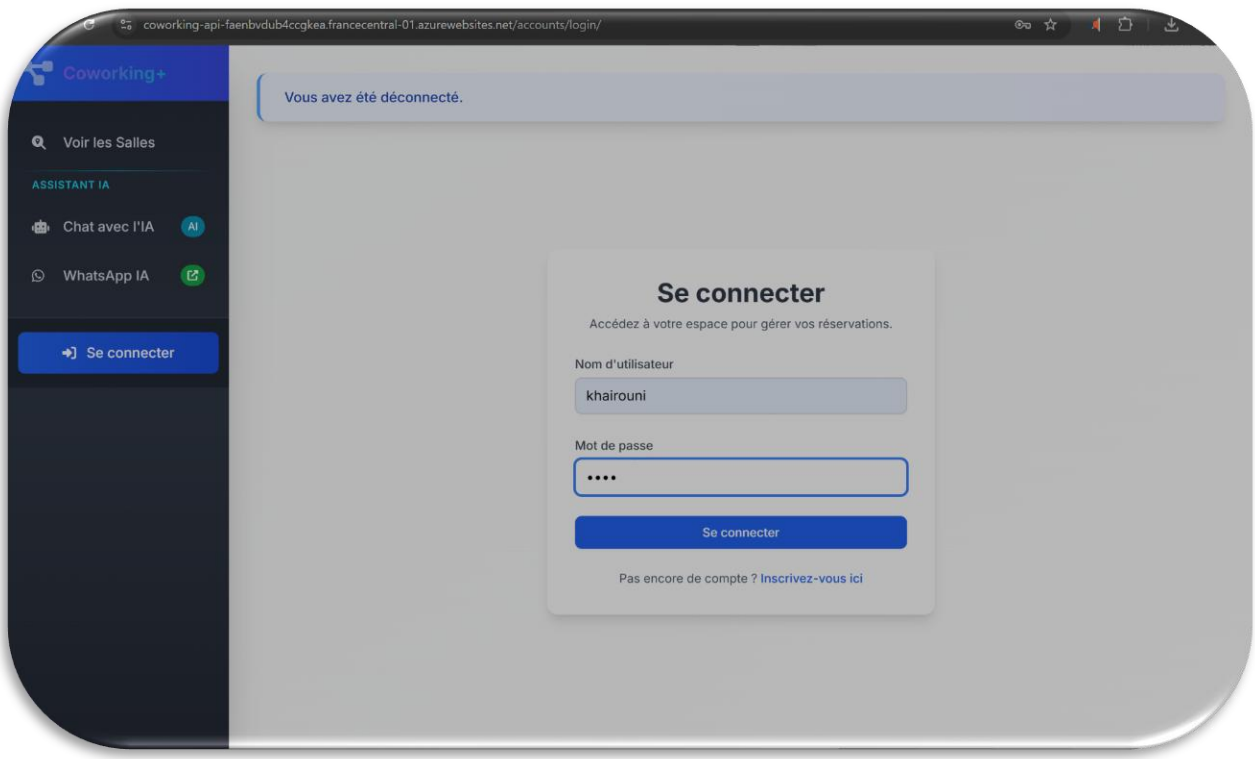
Historique des Réservations Client



Formulaire d'Avis et de Notation



Page de Connexion



Formulaire d'Inscription

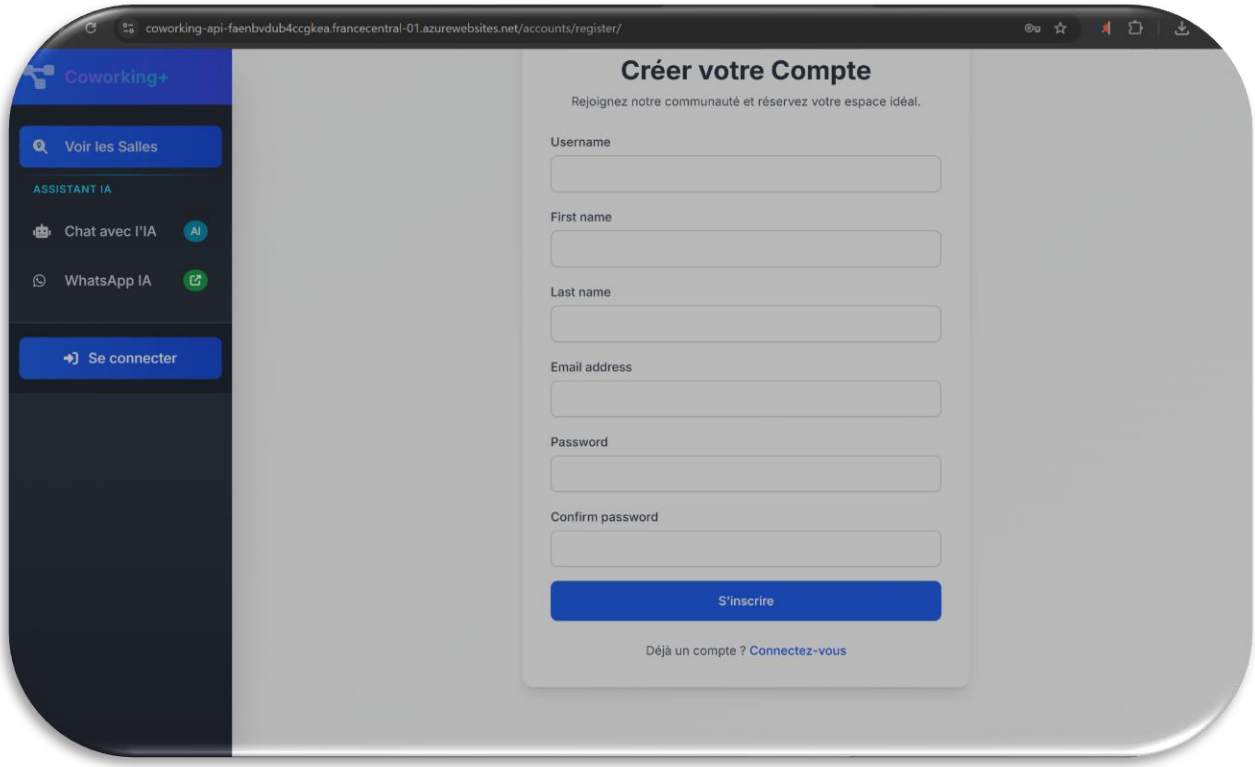
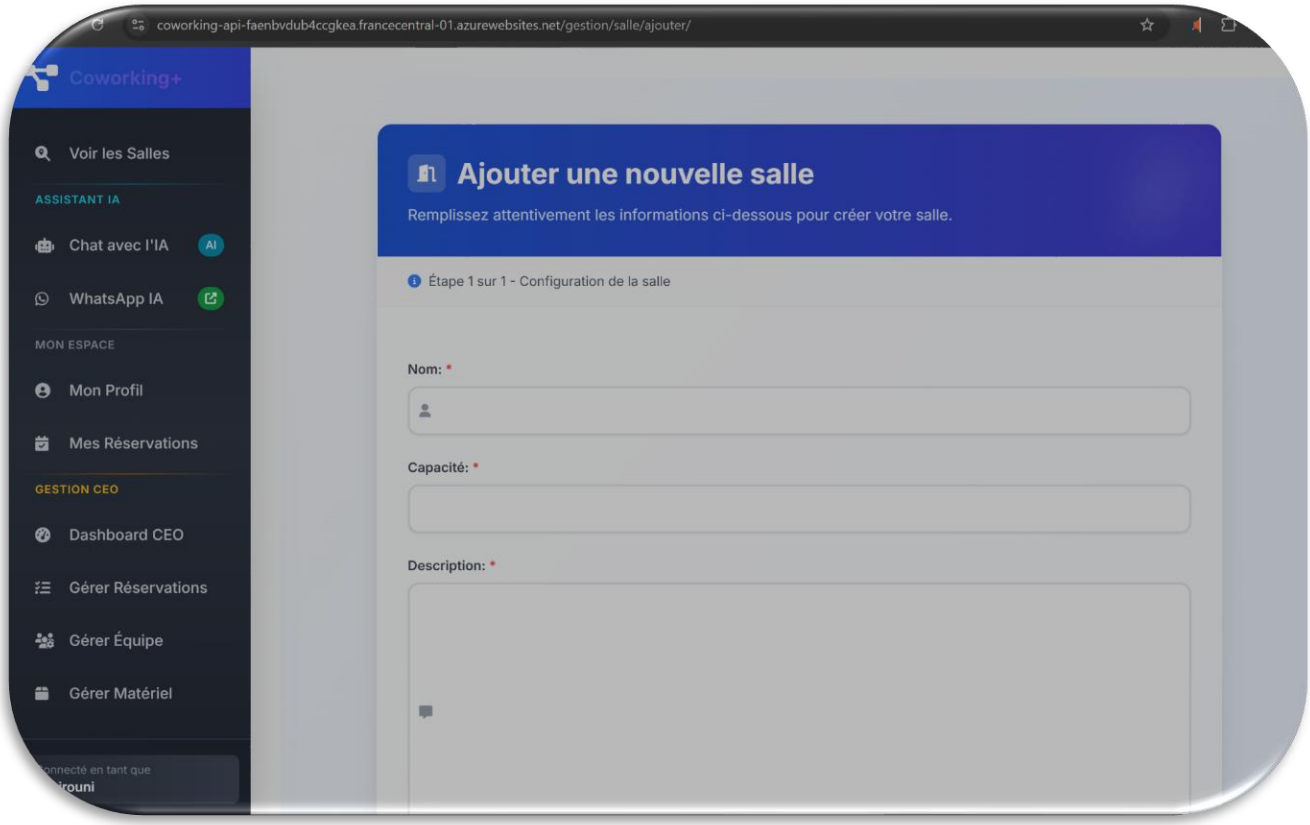


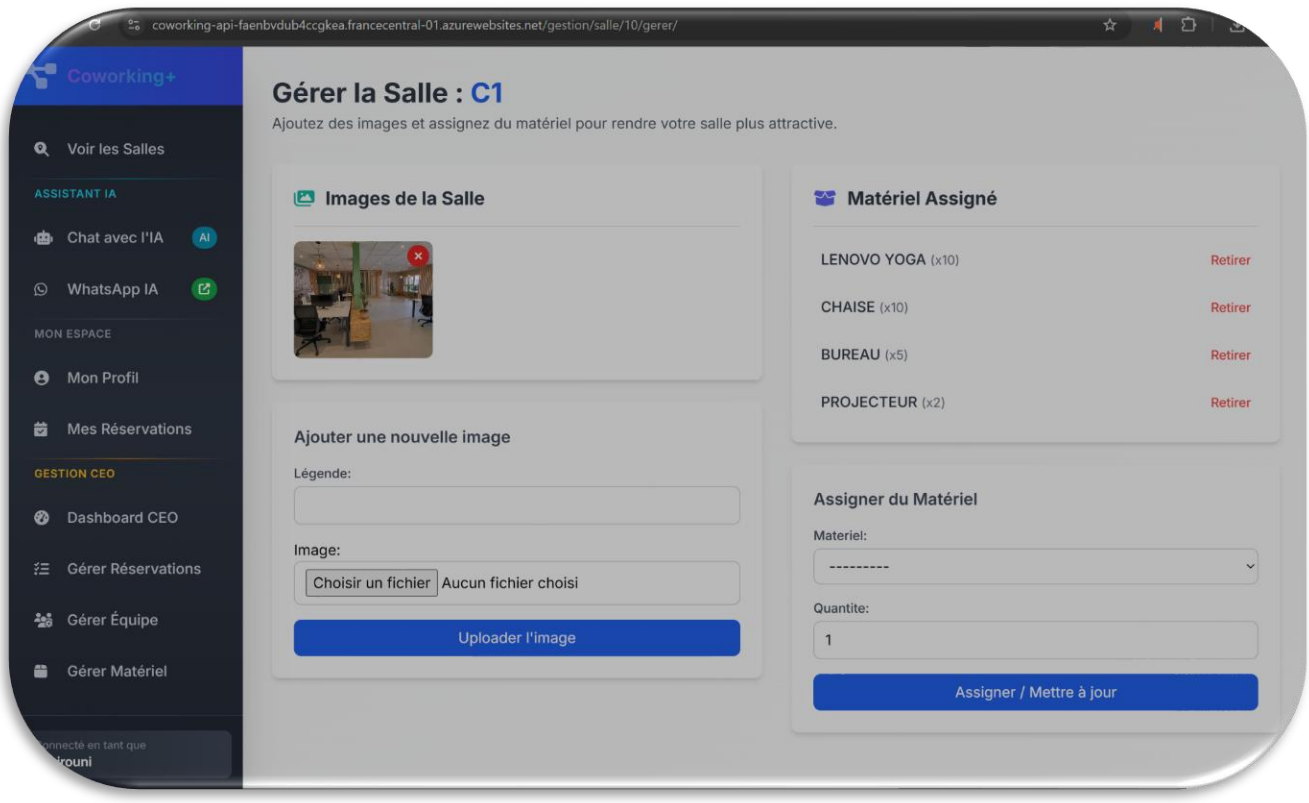
Tableau de Bord du CEO



Formulaire de Création d'une Salle



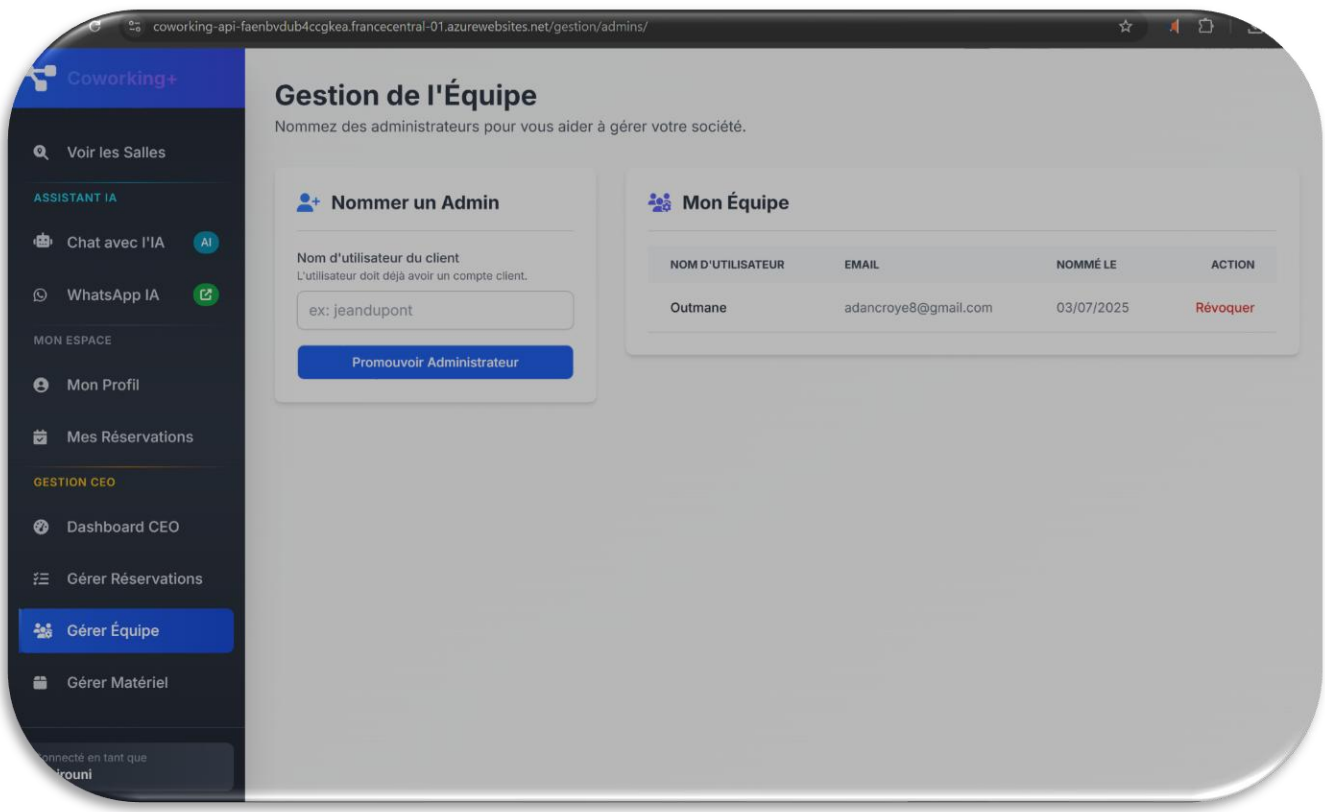
Gestion Détaillée d'une Salle (Images et Matériel)



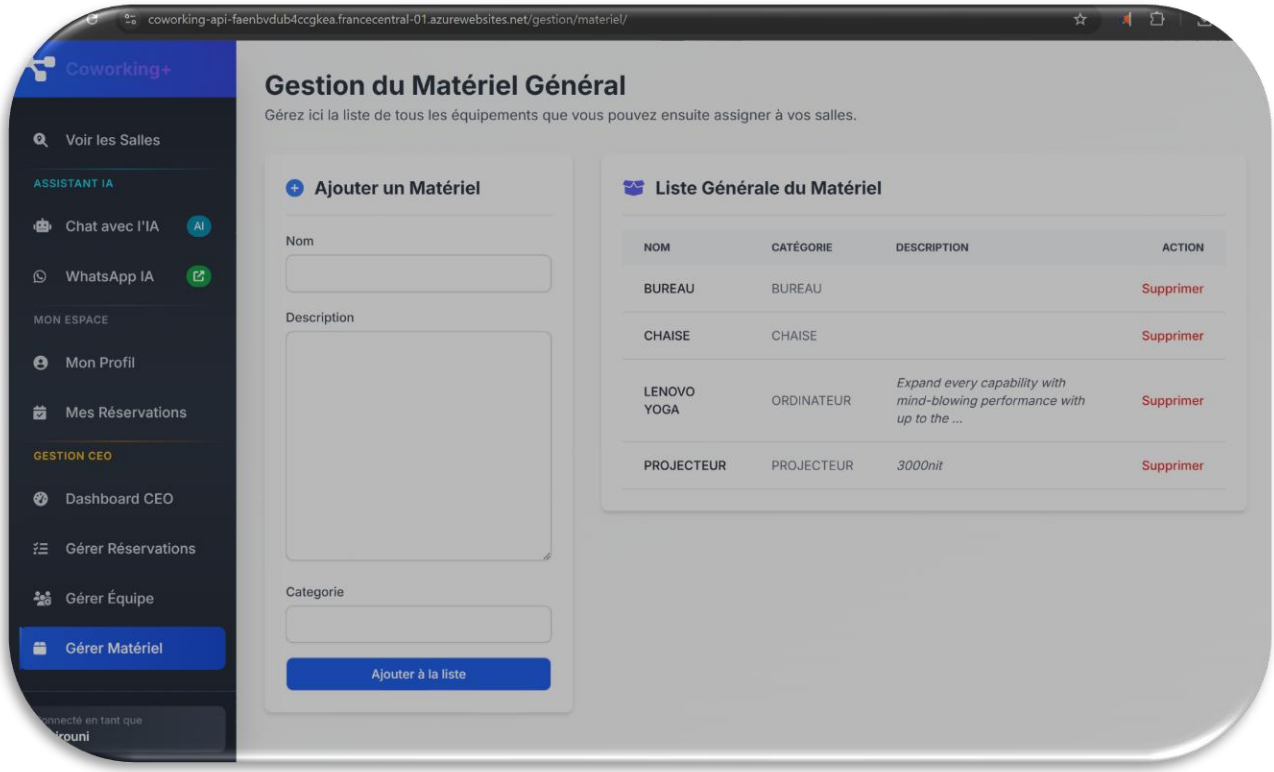
Gestion des Réservations (Vue CEO/Admin)



Gestion de l'Équipe (Administrateurs)



Gestion du Matériel Général



Email d'Annulation avec Code de Remboursement



Confirmation d'annulation de votre réservation

Bonjour ACHRAF,

Nous vous confirmons l'annulation de votre réservation pour la salle **1A** prévue le **06/07/2025** de 09:00 à 12:00.

Conformément à nos conditions, un remboursement d'un montant de **87.00 €** a été effectué sous la forme d'un code de recharge.

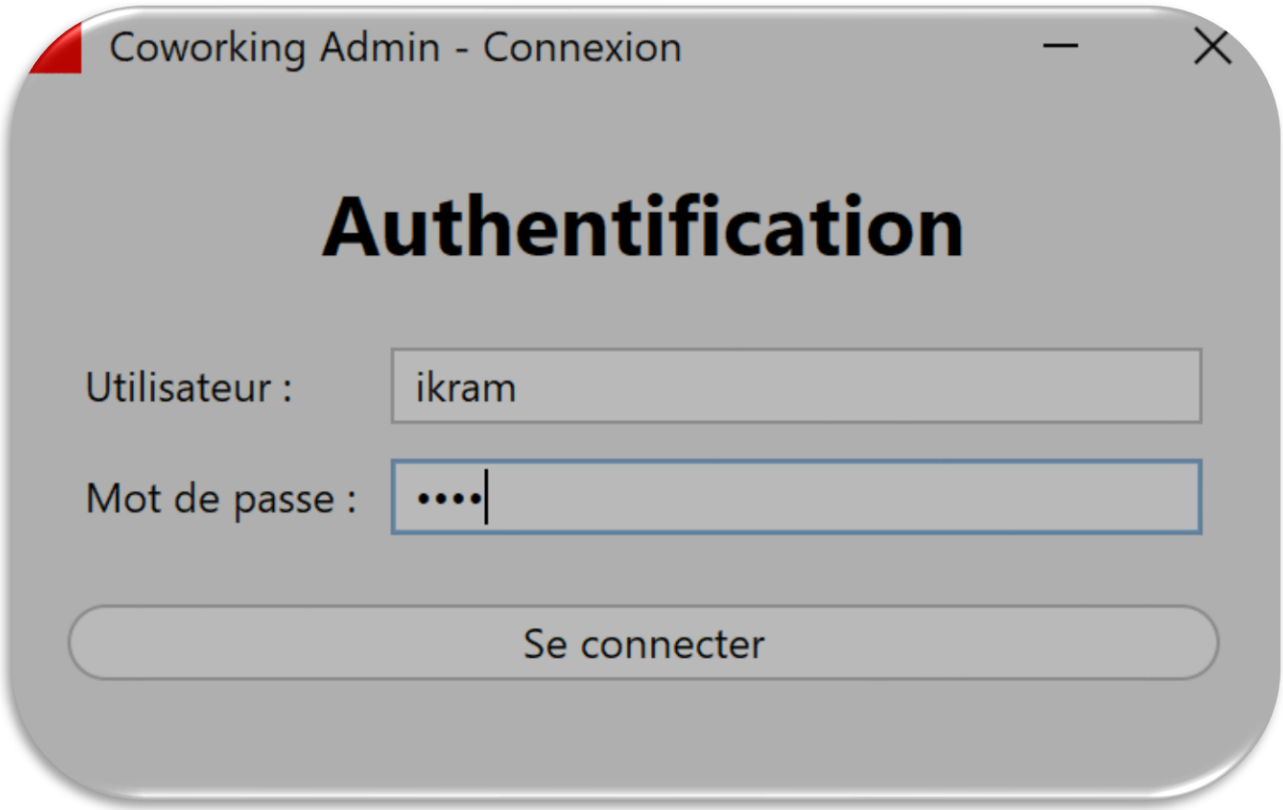
Voici votre code à utiliser lors de votre prochaine visite sur votre profil :

REFUND-41307A61B7

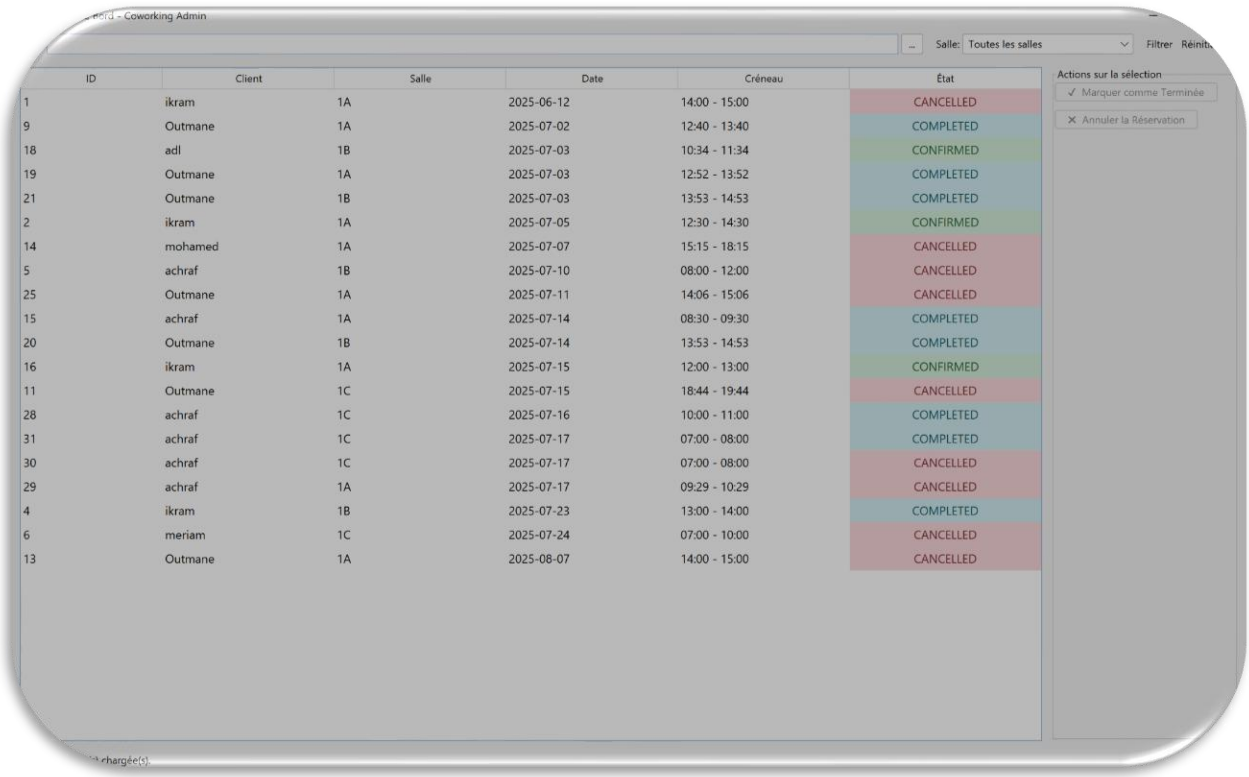
Nous espérons vous revoir bientôt.

L'équipe de notre plateforme de Coworking.

Application Desktop (Java) - Authentification



Application Desktop - Vue de Gestion des Réservations



ID	Client	Salle	Date	Créneau	État
1	ikram	1A	2025-06-12	14:00 - 15:00	CANCELLED
9	Outmane	1A	2025-07-02	12:40 - 13:40	COMPLETED
18	adl	1B	2025-07-03	10:34 - 11:34	CONFIRMED
19	Outmane	1A	2025-07-03	12:52 - 13:52	COMPLETED
21	Outmane	1B	2025-07-03	13:53 - 14:53	COMPLETED
2	ikram	1A	2025-07-05	12:30 - 14:30	CONFIRMED
14	mohamed	1A	2025-07-07	15:15 - 18:15	CANCELLED
5	achraf	1B	2025-07-10	08:00 - 12:00	CANCELLED
25	Outmane	1A	2025-07-11	14:06 - 15:06	CANCELLED
15	achraf	1A	2025-07-14	08:30 - 09:30	COMPLETED
20	Outmane	1B	2025-07-14	13:53 - 14:53	COMPLETED
16	ikram	1A	2025-07-15	12:00 - 13:00	CONFIRMED
11	Outmane	1C	2025-07-15	18:44 - 19:44	CANCELLED
28	achraf	1C	2025-07-16	10:00 - 11:00	COMPLETED
31	achraf	1C	2025-07-17	07:00 - 08:00	COMPLETED
30	achraf	1C	2025-07-17	07:00 - 08:00	CANCELLED
29	achraf	1A	2025-07-17	09:29 - 10:29	CANCELLED
4	ikram	1B	2025-07-23	13:00 - 14:00	COMPLETED
6	meriam	1C	2025-07-24	07:00 - 10:00	CANCELLED
13	Outmane	1A	2025-08-07	14:00 - 15:00	CANCELLED

Conclusion General

Au terme de ce projet, nous avons mené à bien la création d'une plateforme multifacette et entièrement fonctionnelle de réservation d'espaces de coworking. L'ambition initiale était de développer un écosystème numérique capable de répondre aux besoins spécifiques de trois profils d'utilisateurs distincts : le Client final, le Gérant (CEO) de la société de coworking, et l'Administrateur opérationnel. Cet objectif a été atteint avec succès par la mise en œuvre d'une architecture technique moderne et découplée.

Réalisations Techniques et Fonctionnelles

La solution développée se décline en trois composants majeurs :

1. L'Application Web (Backbone du service) : Construite avec le framework Python Django, elle constitue le cœur de la plateforme. Elle offre une expérience utilisateur complète pour :
    - Les Clients, qui disposent d'un cycle de vie complet : inscription avec activation par email, gestion de profil et d'un solde prépayé, consultation des salles, processus de réservation avec paiement, historique et annulation conditionnelle, et système d'évaluation post-visite.
    - Les CEO, qui bénéficient d'un tableau de bord dédié pour la gestion intégrale de leur activité : gestion de leur société, création et modification des salles (CRUD), gestion des images et du matériel associé, et management de leur équipe d'administrateurs.
2. L'API RESTful (Le pont de communication) : Développée avec Django REST Framework, cette API sert d'interface de programmation sécurisée et standardisée. Elle expose les fonctionnalités de gestion nécessaires à l'application de bureau, en assurant l'authentification par tokens (JWT) et la validation des permissions, garantissant qu'un administrateur n'accède qu'aux données de sa propre société.
3. L'Application de Bureau (L'outil opérationnel) : Conçue en Java Swing, elle représente l'outil de travail des administrateurs sur site. Elle consomme l'API pour effectuer des tâches critiques comme la consultation du planning, la finalisation d'une réservation ("check-in") et la gestion des annulations, avec une logique métier spécifique (ex: non-remboursement si moins de 24h).

Intégration Cloud et Choix Technologiques

Un aspect fondamental de ce projet a été la transition d'un environnement de développement local vers une infrastructure cloud professionnelle sur Microsoft Azure. La migration de la base de données SQLite vers une instance managée Azure Database for PostgreSQL et le passage du stockage de fichiers local à Azure Blob Storage ont permis de doter l'application des attributs essentiels de scalabilité, de haute disponibilité et de sécurité, la préparant à un déploiement en production.

Défis Rencontrés et Apprentissages

Le développement a présenté plusieurs défis techniques formateurs, notamment la résolution de dépendances circulaires entre applications Django, la gestion des dépendances de migrations lors du passage à PostgreSQL, et la configuration précise des autorisations d'accès sur Azure. Le succès de leur résolution a renforcé la robustesse de l'architecture finale.

Perspectives d'Évolution

Cette plateforme, bien que complète, offre de nombreuses pistes d'amélioration future :

- Paiement en ligne : Intégration de passerelles de paiement comme Stripe ou PayPal pour la recharge de solde.
- Calendrier Visuel : Développement d'une interface de type calendrier pour visualiser les disponibilités et faire des réservations de manière plus intuitive.
- Tableaux de Bord Analytiques : Fournir aux CEO des graphiques et des statistiques sur leurs revenus et le taux d'occupation de leurs salles.
- Notifications en Temps Réel : via email

En conclusion, ce projet représente une démonstration complète des compétences acquises en ingénierie logicielle.

Auteur :

ACHRAF KHAIROUNI : www.linkedin.com/in/achraf-k-643084284/

IKRAM ROBAII : www.linkedin.com/in/ikram-robaii-90992a335/